UNIT III ACCESS CONTROL AND SECURITY

Syllabus: Network Access Control: Network Access Control, Extensible Authentication Protocol, IEEE 802.1X Port-Based Network Access Control - IP Security - Internet Key Exchange (IKE). Transport-Level Security: Web Security Considerations, Secure Sockets Layer, Transport Layer Security, HTTPS standard, Secure Shell (SSH) application.

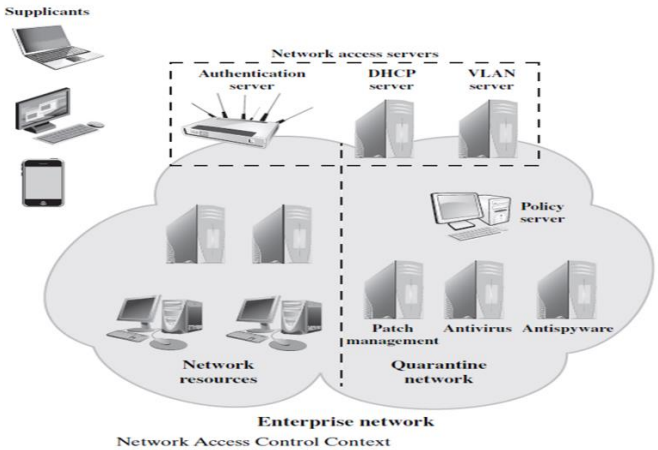
Unit	**Access Control and Security**	**Lecture No**	**C313.3.1**
Topic	Network Access Control: Network Access Control, Extensible Authentication Protocol		
Learning Outcome (LO) At the end of this lecture, students will be able to			**Bloom's Knowledge Level**
LO1	Define NAC and its elements		Remembering
LO2	Describe Extensible Authentication Protocol		Understanding

Network Access Control

- **Network access control (NAC)** is an umbrella term for managing access to a network.
- NAC authenticates users logging into the network and determines what data they can access and actions they can perform.
- NAC also examines the health of the user's computer or mobile device (the endpoints).

Elements of a Network Access Control System

- NAC systems deal with three categories of components:
  - **Access requestor (AR):** The AR is the node that is attempting to access the network and may be any device that is managed by the NAC system, including workstations, servers, printers, cameras, and other IP-enabled devices. Ars are also referred to as **supplicants**, or simply, clients.
  - **Policy server:** Based on the AR's posture and an enterprise's defined policy,the policy server determines what access should be granted. The policy server often relies on backend systems, including antivirus, patch management, or a user directory, to help determine the host's condition.
  - **Network access server (NAS):** The NAS functions as an access control point for users in remote locations connecting to an enterprise's internal network.Also called a **media gateway**, a **remote access server (RAS)**, or a **policy server**, an NAS may include its own authentication services or rely on a separate authentication service from the policy server.
- Figure below is a generic network access diagram.



Network Access Control Context

- The authentication process serves a number of purposes.
- It verifies a supplicant's claimed identity, which enables the policy server to determine what access privileges, if any, the AR may have.
- The authentication exchange may result in the establishment of session keys to enable future secure communication between the supplicant and resources on the enterprise network.
- Typically, the policy server or a supporting server will perform checks on the AR to determine if it should be permitted interactive remote access connectivity.
- These checks should be performed before granting the AR access to the enterprise network.
- Based on the results of these checks, the organization can determine whether the remote computer should be permitted to use interactive remote access.

Network Access Enforcement Methods

- Enforcement methods are the actions that are applied to ARs to regulate access to the enterprise network.
- The following are common **NAC enforcement methods**.

  • **IEEE 802.1X:** This is a link layer protocol that enforces authorization before a port is assigned an IP address. IEEE 802.1X makes use of the Extensible Authentication Protocol for the authentication process.

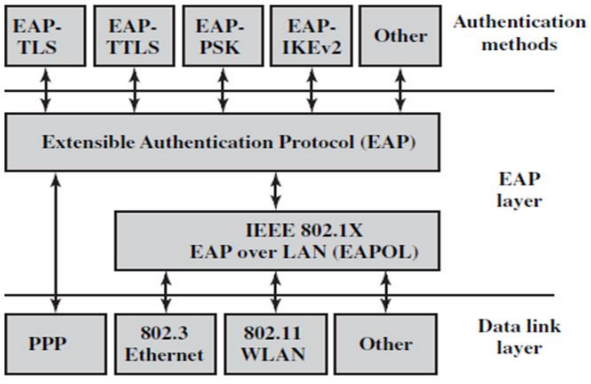
  • **Virtual local area networks (VLANs):** In this approach, the enterprise network, consisting of an interconnected set of LANs, is segmented logically into a number of virtual LANs.

  • **Firewall:** A firewall provides a form of NAC by allowing or denying network traffic between an enterprise host and an external user.

  • **DHCP management:** The Dynamic Host Configuration Protocol (DHCP) is an Internet protocol that enables dynamic allocation of IP addresses to hosts. A DHCP server intercepts DHCP requests and assigns IP addresses instead.

Extensible Authentication Protocol

- The Extensible Authentication Protocol (EAP), defined in RFC 3748, acts as a framework for network access and authentication protocols.
- EAP provides a set of protocol messages that can encapsulate various authentication methods to be used between a client and an authentication server.
- EAP can operate over a variety of network and link level facilities, including point-to-point links, LANs, and other networks, and can accommodate the authentication needs of the various links and networks.
- Figure below illustrates the protocol layers that form the context for EAP.



EAP Layered Context

Authentication Methods

The following are commonly supported EAP methods:

EAP-TLS (EAP Transport Layer Security):

- EAP-TLS (RFC 5216) defines how the TLS protocol can be encapsulated in EAP messages.
- EAP-TLS uses the handshake protocol in TLS, not its encryption method.
- Client and server authenticate each other using digital certificates.
- Client generates a pre-master secret key by encrypting a random number with the server's public key and sends it to the server.
- Both client and server use the pre-master to generate the same secret key.

EAP-TTLS (EAP Tunneled TLS):

- EAP-TTLS is like EAP-TLS, except only the server has a certificate to authenticate itself to the client first.
- As in EAPTLS, a secure connection (the "tunnel") is established with secret keys, but that connection is used to continue the authentication process by authenticating the client and possibly the server again using any EAP method or legacy method such as PAP (Password Authentication Protocol) and CHAP (Challenge-Handshake Authentication Protocol).
- EAP-TTLS is defined in RFC 5281.

EAP-GPSK (EAP Generalized Pre-Shared Key):

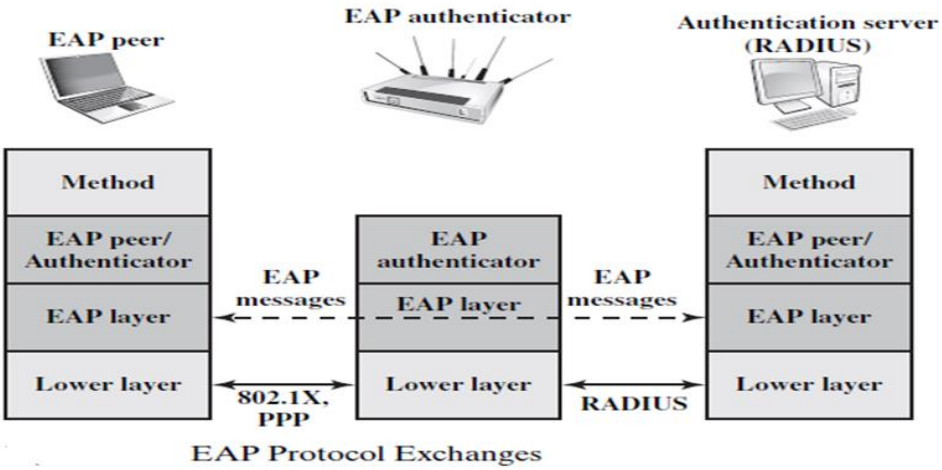
- EAP-GPSK, defined in RFC 5433, is an EAP method for mutual authentication and session key derivation using a Pre-Shared Key (PSK).
- EAP-GPSK specifies an EAP method based on pre-shared keys and employs secret key-based cryptographic algorithms.
- Hence, this method is efficient in terms of message flows and computational costs, but requires the existence of pre-shared keys between each peer and EAP server.

EAP-IKEv2:

- It is based on the Internet Key Exchange protocol version 2 (IKEv2) supports mutual authentication and session key establishment using a variety of methods.
- EAP-TLS is defined in RFC 5106.

EAP Exchanges

- Whatever method is used for authentication, the authentication information and authentication protocol information are carried in EAP messages.
- RFC 3748 defines the goal of the exchange of EAP messages to be successful authentication.
- In the context of RFC 3748, successful authentication is an exchange of EAP messages, as a result of which the authenticator decides to allow access by the peer, and the peer decides to use this access.
- The authenticator's decision typically involves both authentication and authorization aspects; the peer may successfully authenticate to the authenticator, but access may be denied by the authenticator due to policy reasons.
- Figure below indicates a typical arrangement in which EAP is used.

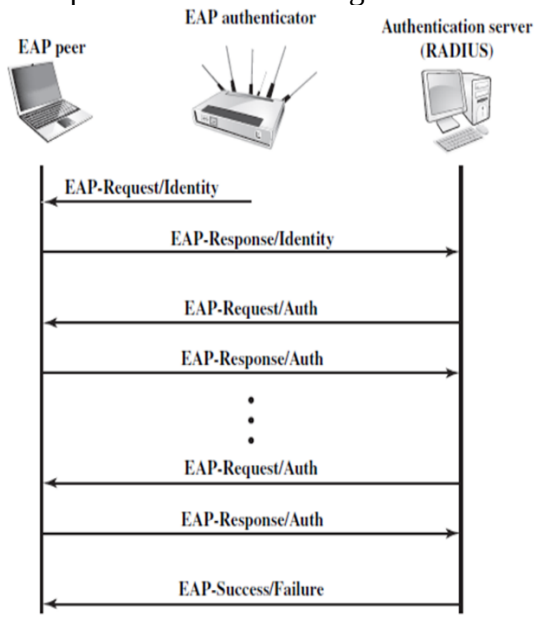


EAP Protocol Exchanges

- The following components are involved:
  - **EAP peer:**
    Client computer that is attempting to access a network.
  - **EAP authenticator:**
    An access point or NAS that requires EAP authentication prior to granting access to a network.
  - **Authentication server:**
    - A server computer that negotiates the use of a specific EAP method with an EAP peer, validates the EAP peer's credentials, and authorizes access to the network.
    - The authentication server is a **Remote Authentication Dial-In User Service (RADIUS) server** that functions as a backend server that can authenticate peers as a service to a number of EAP authenticators.
    - The EAP authenticator then makes the decision of whether to grant access. This is referred to as the EAP **passthrough mode.**
    - Less commonly, the authenticator takes over the role of the EAP server; that is, only two parties are involved in the EAP execution.
    - As a first step, a lower-level protocol, such as PPP (point-to-point protocol) or IEEE 802.1X, is used to connect to the EAP authenticator.
    - The software entity in the EAP peer that operates at this level is referred to as **the supplicant**.
    - EAP messages containing the appropriate information for a chosen EAP method are then exchanged between the EAP peer and the authentication server.
- EAP messages may include the following fields:

  • **Code:** Identifies the Type of EAP message. The codes are Request (1), Response (2), Success (3), and Failure (4).
  • **Identifier:** Used to match Responses with Requests.
  • **Length:** Indicates the length, in octets, of the EAP message, including the Code, Identifier, Length, and Data fields.
  • **Data:** Contains information related to authentication. Typically, the Data field consists of a Type subfield, indicating the type of data carried, and a Type- Data field.
- The Success and Failure messages do not include a Data field.
- The EAP authentication exchange proceeds as follows.

- After a lower-level exchange that established the need for an EAP exchange, the authenticator sends a Request to the peer to request an identity, and the peer sends a Response with the identity information.
- This is followed by a sequence of Requests by the authenticator and Responses by the peer for the exchange of authentication information.
- The information exchanged and the number of Request–Response exchanges needed depend on the authentication method.
- The conversation continues until either (1) the authenticator determines that it cannot authenticate the peer and transmits an EAP Failure or (2) the authenticator determines that successful authentication has occurred and transmits an EAP Success.
- Figure below gives an example of an EAP exchange.



EAP Message Flow in Pass-Through Mode

- The first pair of EAP Request and Response messages is of Type identity, in which the authenticator requests the peer's identity, and the peer returns its claimed identity in the Response message.
- This Response is passed through the authenticator to the authentication server.
- Subsequent EAP messages are exchanged between the peer and the authentication server.

Assessment questions to the lecture

Qn No	Question	Answer	Bloom's Knowledge Level
1	Which component of a NAC system determines access privileges based on enterprise policy? a) Access Requestor (AR) b) Policy Server c) Network Access Server (NAS)	b	Remembering

	d) Supplicant		
2	What is the role of the Extensible Authentication Protocol (EAP)? a) Encrypting all data on the network b) Providing a framework for authentication methods c) Generating VLANs dynamically d) Assigning IP addresses to connected devices	b	Remembering

Students have to prepare answers for the following questions at the end of the lecture

Qn No	Question	Marks	CO	Bloom's Knowledge Level
1	Define NAC and its elements	2	3	Remembering
2	Describe the protocol layers that form the context of Extensible Authentication Protocol (EAP) with a neat diagram. Explain the authentication methods supported by EAP. (April / May 2024)	7	3	Understanding

Reference Book:

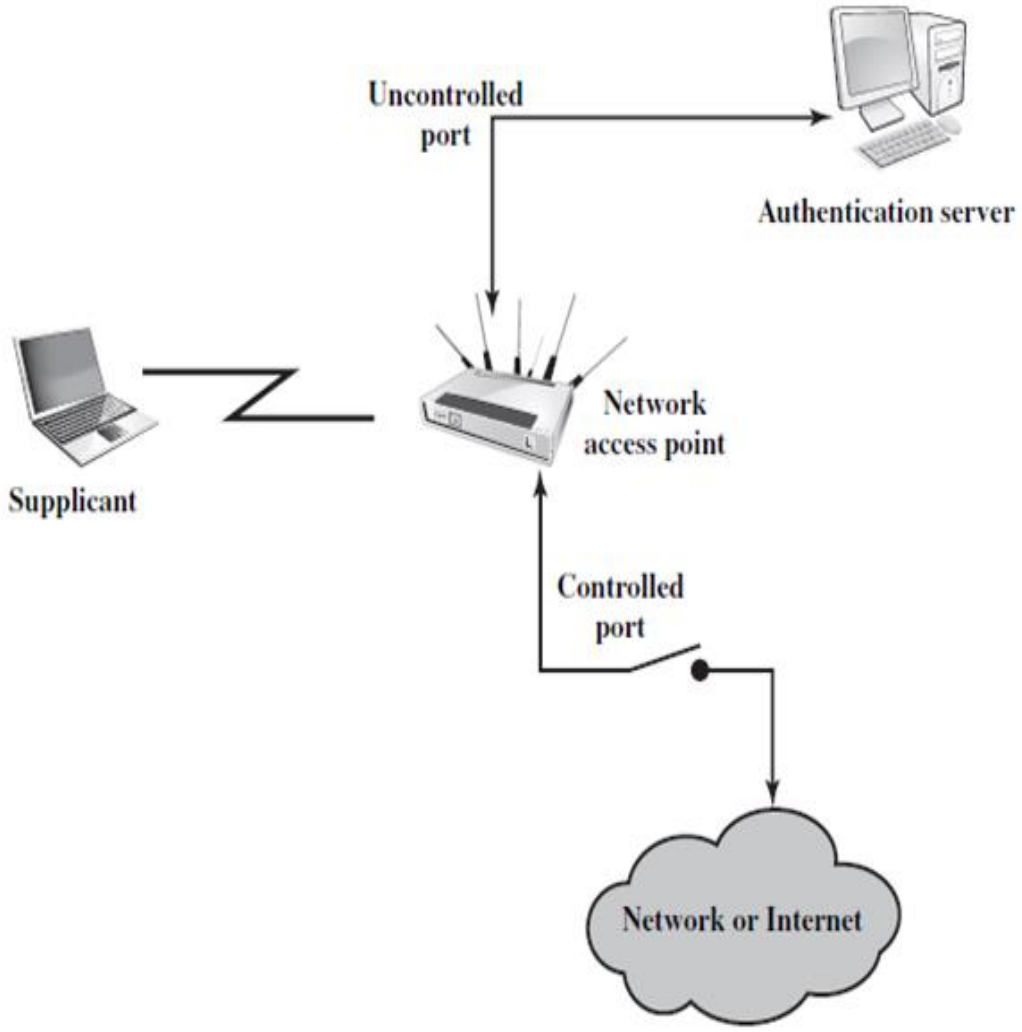
Author(s) Name	Title of the book	Page numbers
William Stallings	Cryptography and Network Security: Principles and Practice, 6th Edition	496-503

Unit	Access Control and Security	Lecture No	C313.3.2
Topic	IEEE 802.1X Port-Based Network Access Control - IP Security - Internet Key Exchange (IKE)		

Learning Outcome (LO) At the end of this lecture, students will be able to		**Bloom's Knowledge Level**
LO1	Describe IEEE 802.1X Port-Based Network Access Control	Understanding
LO2	Define Internet Key Exchange (IKE).	Remembering
LO3	Explain the working of IP Security (IPSec) and the role of Internet Key Exchange (IKE) in securing IP communication.	Understanding

IEEE 802.1X Port-Based Network Access Control

- IEEE 802.1X Port-Based Network Access Control was designed to provide access control functions for LANs.



802.1X Access Control

- Table briefly defines key terms used in the IEEE 802.11 standard.

Terminology Related to IEEE 802.1X

Authenticator
An entity at one end of a point-to-point LAN segment that facilities authentication of the entity to the other end of the link.

Authentication exchange
The two-party conversation between systems performing an authentication process.

Authentication process
The cryptographic operations and supporting data frames that perform the actual authentication.

Authentication server (AS)
An entity that provides an authentication service to an authenticator. This service determines, from the credentials provided by supplicant, whether the supplicant is authorized to access the services provided by the system in which the authenticator resides.

Authentication transport
The datagram session that actively transfers the authentication exchange between two systems.

Bridge port
A port of an IEEE 802.1D or 802.1Q bridge.

Edge port
A bridge port attached to a LAN that has no other bridges attached to it.

Network access port
A point of attachment of a system to a LAN. It can be a physical port, such as a single LAN MAC attached to a physical LAN segment, or a logical port, for example, an IEEE 802.11 association between a station and an access point.

Port access entity (PAE)
The protocol entity associated with a port. It can support the protocol functionality associated with the authenticator, the supplicant, or both.

Supplicant
An entity at one end of a point-to-point LAN segment that seeks to be authenticated by an authenticator attached to the other end of that link.

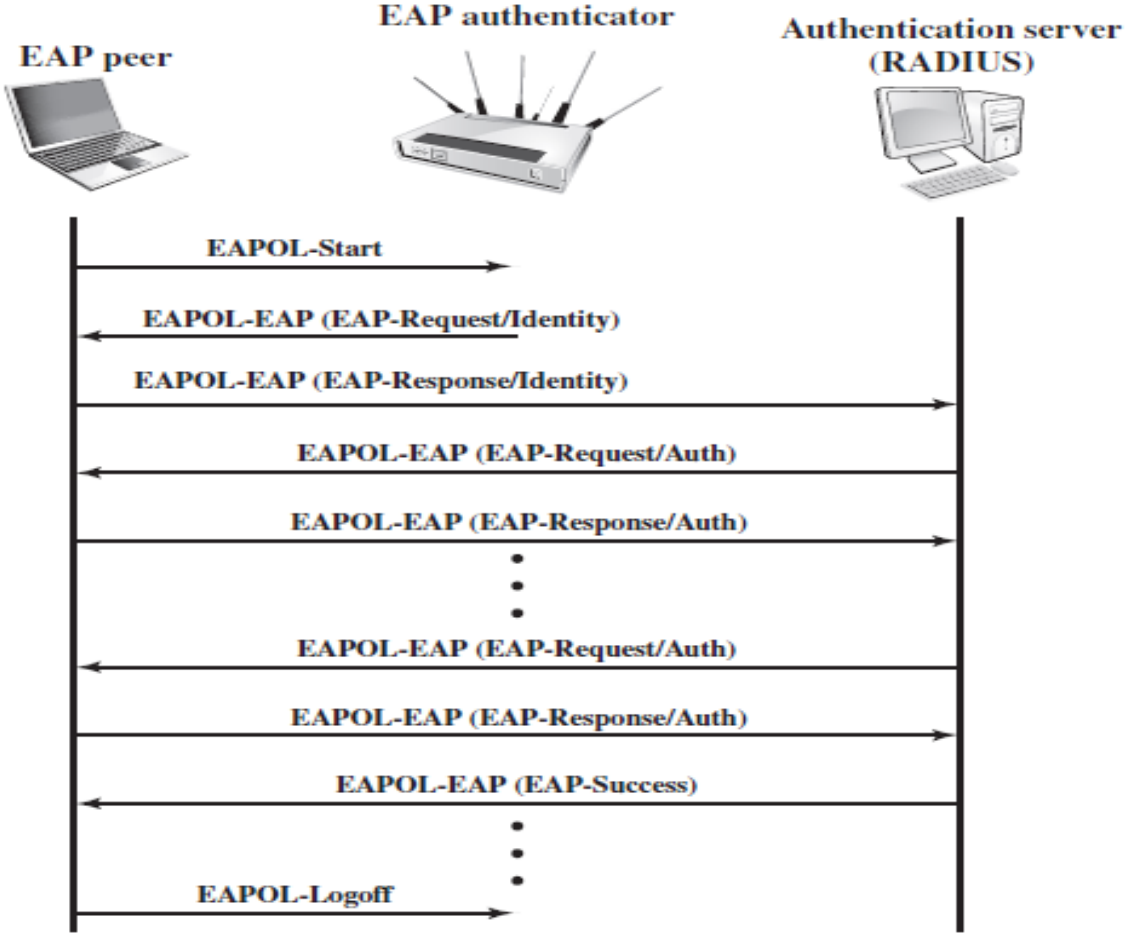
- The terms *supplicant*, *network access point*, and *authentication server* correspond to the EAP terms *peer*, *authenticator*, and *authentication server*, respectively.
- Until the AS authenticates a supplicant (using an authentication protocol), the authenticator only passes control and authentication messages between the supplicant and the AS; the 802.1X control channel is unblocked, but the 802.11 data channel is blocked.
- Once a supplicant is authenticated and keys are provided, the authenticator can forward data from the supplicant, subject to predefined access control limitations for the supplicant to the network.
- Under these circumstances, the data channel is unblocked.
- Ports are logical entities defined within the authenticator and refer to physical network connections.
- Each logical port is mapped to one of these two types of physical ports.
- An uncontrolled port allows the exchange of protocol data units (PDUs) between the supplicant and the AS, regardless of the authentication state of the supplicant.
- A controlled port allows the exchange of PDUs between a supplicant and other systems on the network only if the current state of the supplicant authorizes such an exchange.

- The essential element defined in 802.1X is a protocol known as EAPOL (EAP over LAN). EAPOL operates at the network layers and makes use of an IEEE 802 LAN, such as Ethernet or Wi-Fi, at the link level.
- EAPOL enables a supplicant to communicate with an authenticator and supports the exchange of EAP packets for authentication.
- The most common EAPOL packets are listed in Table below.

Common EAPOL Frame Types

Frame Type	Definition
EAPOL-EAP	Contains an encapsulated EAP packet.
EAPOL-Start	A supplicant can issue this packet instead of waiting for a challenge from the authenticator.
EAPOL-Logoff	Used to return the state of the port to unauthorized when the supplicant if finished using the network.
EAPOL-Key	Used to exchange cryptographic keying information.

- When the supplicant first connects to the LAN, it does not know the MAC address of the authenticator.
- Actually, it doesn't know whether there is an authenticator present at all.
- By sending an **EAPOL-Start** packet to a special group-multicast address reserved for IEEE 802.1X authenticators, a supplicant can determine whether an authenticator is present and let it know that the supplicant is ready.
- In many cases,the authenticator will already be notified that a new device has connected from some hardware notification.
- For example, a hub knows that a cable is plugged in before the device sends any data. In this case the authenticator may preempt the Start message with its own message.
- In either case the authenticator sends an EAPRequest Identity message encapsulated in an **EAPOL-EAP** packet.
- The EAPOLEAP is the EAPOL frame type used for transporting EAP packets.
- The authenticator uses the **EAP-Key** packet to send cryptographic keys to the supplicant once it has decided to admit it to the network.
- The **EAP-Logoff** packet type indicates that the supplicant wishes to be disconnected from the network.
- The **EAPOL packet format includes the following fields**:
  • **Protocol version:** version of EAPOL.
  • **Packet type:** indicates start, EAP, key, logoff, etc.
  • **Packet body length:** If the packet includes a body, this field indicates the body length.
  • **Packet body:** The payload for this EAPOL packet. An example is an EAP packet.
- Figure below shows an example of exchange using EAPOL.



Example Timing Diagram for IEEE 802.1X

IP Security

- IP Security (IPSec) is a suite of protocols used to secure Internet Protocol (IP) communications by authenticating and encrypting each IP packet within a communication session.
- To provide security, the Internet Architecture Board (IAB) included authentication and encryption as necessary security features in the next-generation IP, which has been issued as IPv6.

Applications of IPsec

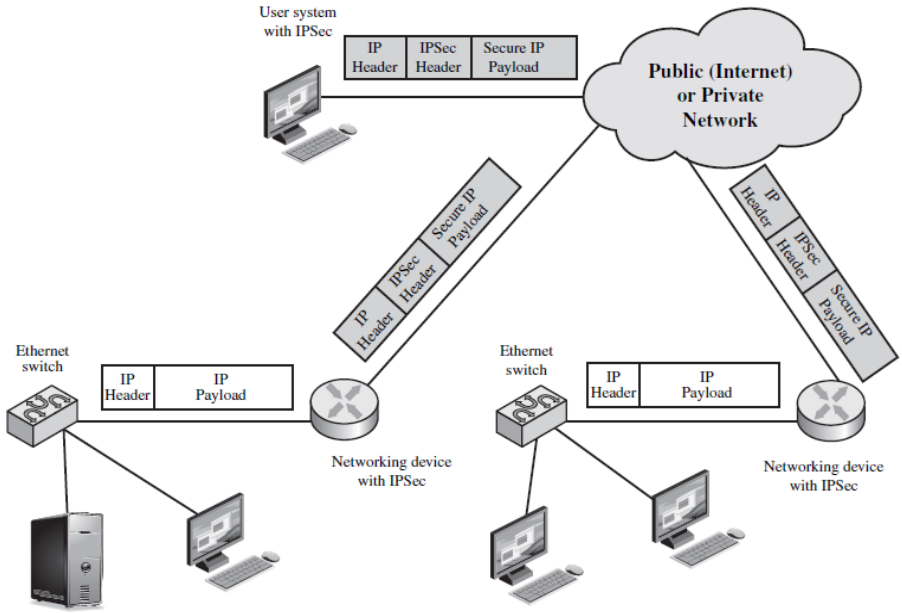
- IPsec provides the capability to secure communications across a LAN, across private and public WANs, and across the Internet. Examples of its use include:

  • **Secure branch office connectivity over the Internet:** A company can build a secure virtual private network over the Internet or over a public WAN.
  • **Secure remote access over the Internet:** An end user whose system is equipped with IP security protocols can make a local call to an Internet Service Provider (ISP) and gain secure access to a company network.
  • **Establishing extranet and intranet connectivity with partners:** IPsec can be used to secure communication with other organizations, ensuring authentication and

confidentiality and providing a key exchange mechanism.

• **Enhancing electronic commerce security:** IPsec guarantees that all traffic designated by the network administrator is both encrypted and authenticated, adding an additional layer of security to whatever is provided at the application layer.

- Figure below is a typical scenario of IPsec usage.



An IP Security Scenario

Benefits of IPsec

• When IPsec is implemented in a firewall or router, it provides strong security that can be applied to all traffic crossing the perimeter.

• IPsec in a firewall is resistant to bypass if all traffic from the outside must use IP and the firewall is the only means of entrance from the Internet into the organization.

• IPsec is below the transport layer (TCP, UDP) and so is transparent to applications. There is no need to change software on a user or server system when IPsec is implemented in the firewall or router.

• IPsec can be transparent to end users.

Routing Applications

IPsec can assure that

• A router advertisement (a new router advertises its presence) comes from an authorized router.

• A neighbor advertisement (a router seeks to establish or maintain a neighbor relationship with a router in another routing domain) comes from an authorized router.

• A redirect message comes from the router to which the initial IP packet was sent.

• A routing update is not forged.

IPsec Documents

The documents can be categorized into the following groups.

• **Architecture:** Covers the general concepts, security requirements, definitions,and mechanisms defining IPsec technology. The current specification is RFC 4301, *Security Architecture for the Internet Protocol*.

• **Authentication Header (AH):** AH is an extension header to provide message authentication. The current specification is RFC 4302, *IP Authentication Header*. Because message authentication is provided by ESP, the use of AH is deprecated. It is included in IPsecv3 for backward compatibility but should not be used in new applications.

• **Encapsulating Security Payload (ESP):** ESP consists of an encapsulating header and trailer used to provide encryption or combined encryption/authentication. The current specification is RFC 4303, *IP Encapsulating Security Payload (ESP)*.

• **Internet Key Exchange (IKE):** This is a collection of documents describing the key management schemes for use with IPsec. The main specification is RFC 5996, *Internet Key Exchange (IKEv2) Protocol*, but there are a number of related RFCs.

IPsec Services

- IPsec provides security services at the IP layer by enabling a system to select required security protocols, determine the algorithm(s) to use for the service(s), and put in place any cryptographic keys required to provide the requested services.
- Two protocols are used to provide security: an authentication protocol designated by the header of the protocol, Authentication Header (AH); and a combined encryption/ authentication protocol designated by the format of the packet for that protocol, Encapsulating Security Payload (ESP).
- RFC 4301 lists the following services:

  • Access control

  • Connectionless integrity

  • Data origin authentication

  • Rejection of replayed packets (a form of partial sequence integrity)

  • Confidentiality (encryption)

  • Limited traffic flow confidentiality

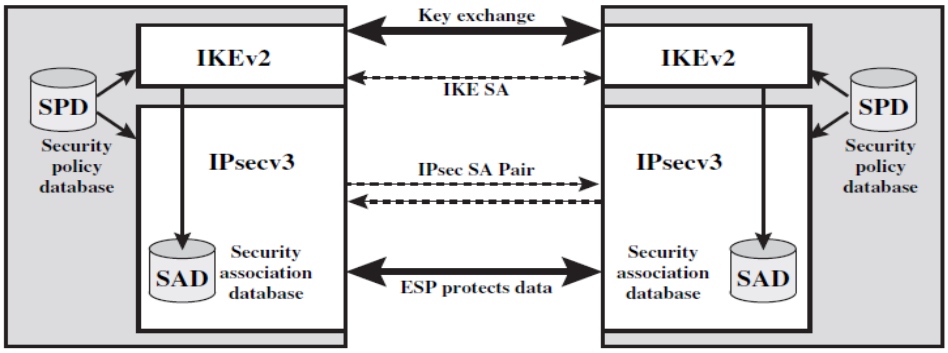
Transport and Tunnel Modes

- Both AH and ESP support two modes of use: transport and tunnel mode.
  - ***Transport Mode*** Transport mode provides protection primarily for upper-layer protocols. That is, transport mode protection extends to the payload of an IP packet.
  - ***Tunnel Mode*** Tunnel mode provides protection to the entire IP packet. To achieve this, after the AH or ESP fields are added to the IP packet, the entire packet plus security fields is treated as the payload of new outer IP packet with a new outer IP header. The entire original, inner, packet travels through a tunnel from one point of an IP network to another; no routers along the way are able to examine the inner IP header.

	Transport Mode SA	Tunnel Mode SA
AH	Authenticates IP payload and selected portions of IP header and IPv6 extension headers.	Authenticates entire inner IP packet (inner header plus IP payload) plus selected portions of outer IP header and outer IPv6 extension headers.
ESP	Encrypts IP payload and any IPv6 extension headers following the ESP header.	Encrypts entire inner IP packet.
ESP with Authentication	Encrypts IP payload and any IPv6 extension headers following the ESP header. Authenticates IP payload but not IP header.	Encrypts entire inner IP packet. Authenticates inner IP packet.

IP security policy

- IPsec policy is determined primarily by the interaction of two databases, the **security association database (SAD)** and the **security policy database (SPD)**. Figure illustrates the relevant relationships.



IPsec Architecture

Security Associations

- A key concept that appears in both the authentication and confidentiality mechanisms for IP is the security association (SA). An association is a one-way logical connection between a sender and a receiver that affords security services to the traffic carried on it.
- A security association is uniquely identified by three parameters.
  - **Security Parameters Index (SPI):** A 32-bit unsigned integer assigned to this SA and having local significance only.
  - **IP Destination Address:** This is the address of the destination endpoint of the SA, which may be an end-user system or a network system such as a firewall or router.
  - **Security Protocol Identifier:** This field from the outer IP header indicates whether the association is an AH or ESP security association.

Security Association Database

- In each IPsec implementation, there is a nominal2 Security Association Database that defines the parameters associated with each SA.
- A security association is normally defined by the following parameters in an SAD entry.

- **Security Parameter Index:** A 32-bit value selected by the receiving end of an SA to uniquely identify the SA.
- **Sequence Number Counter:** A 32-bit value used to generate the Sequence Number field in AH or ESP headers.
- **Sequence Counter Overflow:** A flag indicating whether overflow of the Sequence Number Counter should generate an auditable event and prevent further transmission of packets on this SA (required for all implementations).
- **Anti-Replay Window:** Used to determine whether an inbound AH or ESP packet is a replay.
- **AH Information:** Authentication algorithm, keys, key lifetimes, and related parameters being used with AH.
- **ESP Information:** Encryption and authentication algorithm, keys, initialization values, key lifetimes, and related parameters being used with ESP.
- **Lifetime of this Security Association:** A time interval or byte count after which an SA must be replaced with a new SA (and new SPI) or terminated, plus an indication of which of these actions should occur.
- **IPsec Protocol Mode:** Tunnel, transport, or wildcard.
- **Path MTU:** Any observed path maximum transmission unit and aging variables.

Security Policy Database

- The relevant IPsec documents called *selectors*. In effect, these selectors are used to filter outgoing traffic in order to map it into a particular SA. Outbound processing obeys the following general sequence for each IP packet.
  1. Compare the values of the appropriate fields in the packet (the selector fields) against the SPD to find a matching SPD entry, which will point to zero or more SAs.
  2. Determine the SA if any for this packet and its associated SPI.
  3. Do the required IPsec processing (i.e., AH or ESP processing).

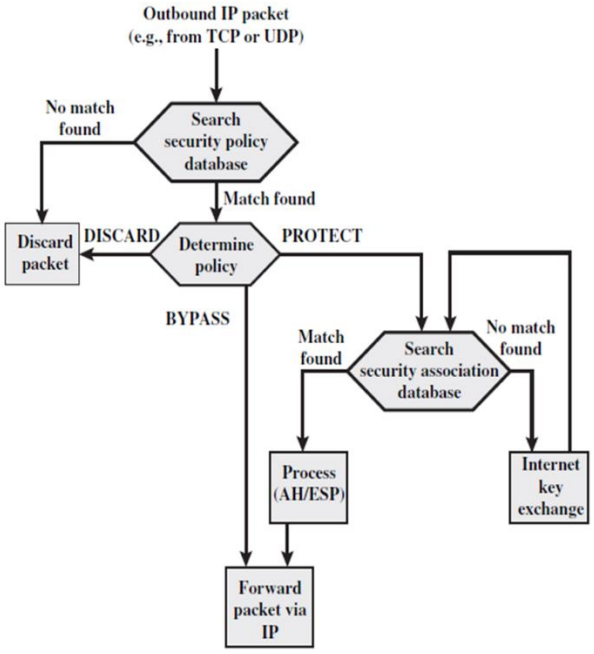
The following selectors determine an SPD entry:

- **Remote IP Address:** This may be a single IP address, an enumerated list or range of addresses, or a wildcard (mask) address.
- **Local IP Address:** This may be a single IP address, an enumerated list or range of addresses, or a wildcard (mask) address.
- **Next Layer Protocol:** This is an individual protocol number, ANY, or for IPv6 only, OPAQUE. If AH or ESP is used, then this IP protocol header immediately precedes the AH or ESP header in the packet.
- **Name:** A user identifier from the operating system.
- **Local and Remote Ports:** These may be individual TCP or UDP port values, an enumerated list of ports, or a wildcard port.

IP Traffic Processing

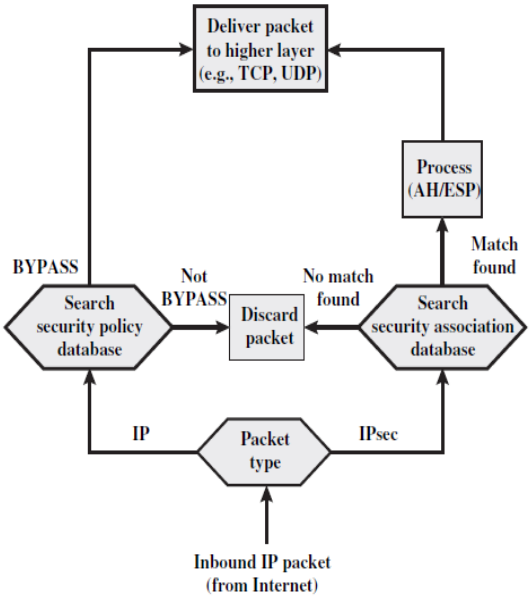
- IPsec is executed on a packet-by-packet basis.

- When IPsec is implemented, each outbound IP packet is processed by the IPsec logic before transmission, and each inbound packet is processed by the IPsec logic after reception and before passing the packet contents on to the next higher layer (e.g., TCP or UDP).
- ***Outbound Packets*** Figure below highlights the main elements of IPsec processing for outbound traffic.



Processing Model for Outbound Packets

- ***Inbound Packets*** Figure below highlights the main elements of IPsec processing for inbound traffic. An incoming IP packet triggers the IPsec processing.



Processing Model for Inbound Packets

Internet Key Exchange

- The key management portion of IPsec involves the determination and distribution of secret keys.
- A typical requirement is four keys for communication between two applications: transmit and receive pairs for both integrity and confidentiality.
- The IPsec Architecture document mandates support for two types of key management:
  - Manual: A system administrator manually configures each system with its own keys and with the keys of other communicating systems. This is practical for small, relatively static environments.
  - Automated: An automated system enables the on-demand creation of keys for SAs and facilitates the use of keys in a large distributed system with an evolving configuration.
- The default automated key management protocol for IPsec is referred to as ISAKMP/Oakley and consists of the following elements:
  - Oakley Key Determination Protocol: Oakley is a key exchange protocol based on the Diffie-Hellman algorithm but providing added security. Oakley is generic in that it does not dictate specific formats.
  - Internet Security Association and Key Management Protocol (ISAKMP): ISAKMP provides a framework for Internet key management and provides the specific protocol support, including formats, for negotiation of security attributes.

Key Determination Protocol

- IKE key determination is a refinement of the Diffie-Hellman key exchange algorithm.
- There is prior agreement on two global parameters: q, a large prime number; and a, a primitive root of q.
- A selects a random integer X_A as its private key and transmits to B its public key $Y_A = \alpha^{X_A} \bmod q$. Similarly, B selects a random integer X_B as its private key and transmits to A its public key $Y_B = \alpha^{X_B} \bmod q$.
- Each side can now compute the secret session key:

$$K = (Y_B)^{X_A} \bmod q = (Y_A)^{X_B} \bmod q = \alpha^{X_A X_B} \bmod q$$

- The Diffie-Hellman algorithm has two attractive features:
  - Secret keys are created only when needed.
  - The exchange requires no pre-existing infrastructure other than an agreement on the global parameters.

Features of IKE key determination

The IKE key determination algorithm is characterized by five important features:

- It employs a mechanism known as cookies to thwart clogging attacks.
- It enables the two parties to negotiate a group; this, in essence, specifies the global parameters of the Diffie-Hellman key exchange.
- It uses nonces to ensure against replay attacks.

- It enables the exchange of Diffie-Hellman public key values.
- It authenticates the Diffie-Hellman exchange to thwart man-in-the-middle attacks.

IKE mandates that cookie generation satisfy three basic requirements:

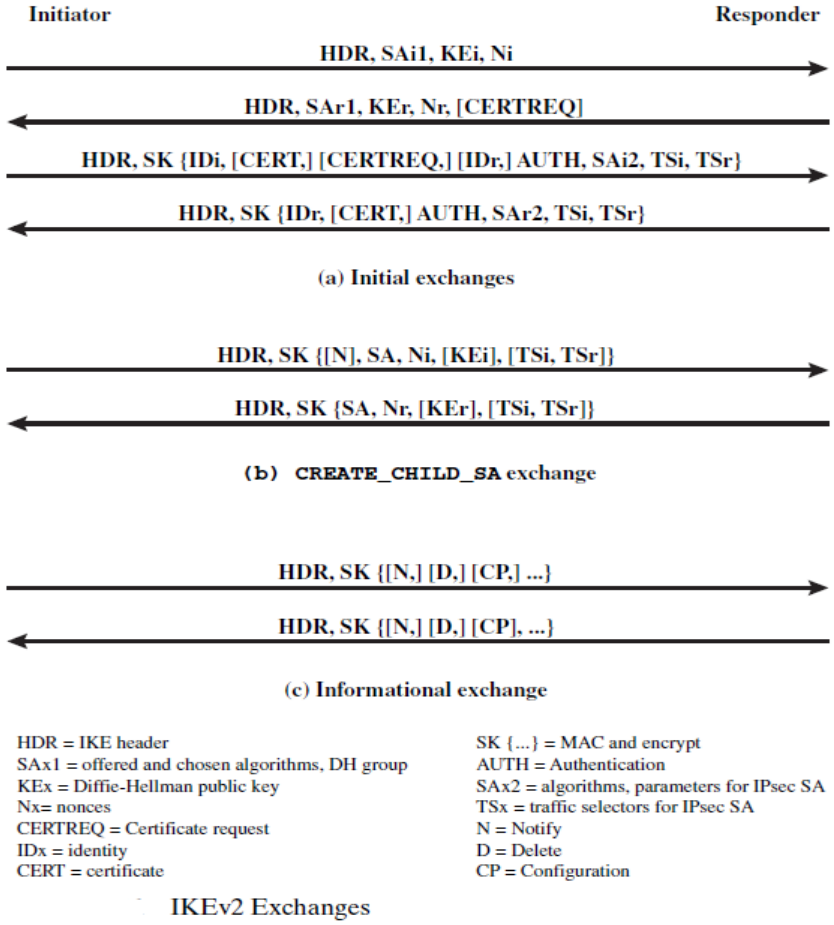
1. The cookie must depend on the specific parties. This prevents an attacker from obtaining a cookie using a real IP address and UDP port and then using it to swamp the victim with requests from randomly chosen IP addresses or ports.
2. It must not be possible for anyone other than the issuing entity to generate cookies that will be accepted by that entity. This implies that the issuing entity will use local secret information in the generation and subsequent verification of a cookie.
3. The cookie generation and verification methods must be fast to thwart attacks intended to sabotage processor resources.

Three different authentication methods can be used with IKE key determination:

- **Digital signatures**: The exchange is authenticated by signing a mutually obtainable hash; each party encrypts the hash with its private key. The hash is generated over important parameters, such as user IDs and nonces.
- **Public-key encryption**: The exchange is authenticated by encrypting parameters such as IDs and nonces with the sender's private key.
- **Symmetric-key encryption**: A key derived by some out-of-band mechanism can be used to authenticate the exchange by symmetric encryption of exchange parameters.

IKE v2 Exchanges

- The IKEv2 protocol involves the exchange of messages in pairs.
- The first two pairs of exchanges are referred to as the initial exchanges (Figure a).

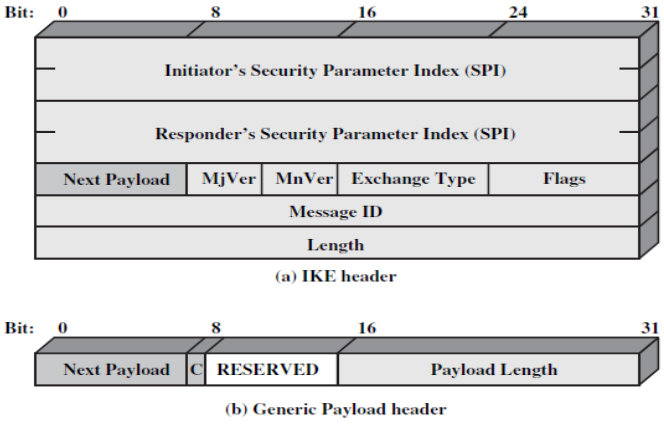


IKEv2 Exchanges

- The **CREATE_CHILD_SA exchange** can be used to establish further Sas for protecting traffic.
- The **informational exchange** is used to exchange management information, IKEv2 error messages, and other notifications.

Header and Payload Formats

Figure below shows the header format for an IKE message. It consists of the following fields.



IKE Formats

- **Initiator SPI (64 bits):** A value chosen by the initiator to identify a unique IKE security association (SA).

- **Responder SPI (64 bits):** A value chosen by the responder to identify a unique IKE SA.
- **Next Payload (8 bits):** Indicates the type of the first payload in the message;
- **Major Version (4 bits):** Indicates major version of IKE in use.
- **Minor Version (4 bits):** Indicates minor version in use.
- **Exchange Type (8 bits):** Indicates the type of exchange;
- **Flags (8 bits):** Indicates specific options set for this IKE exchange.
- **Message ID (32 bits):** Used to control retransmission of lost packets and matching of requests and responses.
- **Length (32 bits):** Length of total message (header plus all payloads) in octets.

IKE Payload Types

- All IKE payloads begin with the same generic payload header shown in Figure b.
- Table below summarizes the payload types defined for IKE and lists the fields, or parameters, that are part of each payload.

IKE Payload Types

Type	Parameters
Security Association	Proposals
Key Exchange	DH Group #, Key Exchange Data
Identification	ID Type, ID Data
Certificate	Cert Encoding, Certificate Data
Certificate Request	Cert Encoding, Certification Authority
Authentication	Auth Method, Authentication Data
Nonce	Nonce Data
Notify	Protocol-ID, SPI Size, Notify Message Type, SPI, Notification Data
Delete	Protocol-ID, SPI Size, # of SPIs, SPI (one or more)
Vendor ID	Vendor ID
Traffic Selector	Number of TSs, Traffic Selectors
Encrypted	IV, Encrypted IKE payloads, Padding, Pad Length, ICV
Configuration	CFG Type, Configuration Attributes
Extensible Authentication Protocol	EAP Message

- These elements are formatted as substructures within the payload as follows.

  • **Proposal:** This substructure includes a proposal number, a protocol ID (AH, ESP, or IKE), an indicator of the number of transforms, and then a transform substructure.
  • **Transform:** Different protocols support different transform types.
  • **Attribute:** Each transform may include attributes that modify or complete the specification of the transform.
- The **Key Exchange payload** can be used for a variety of key exchange techniques, including Oakley, Diffie-Hellman, and the RSA-based key exchange used by PGP.
- The **Identification payload** is used to determine the identity of communicating peers and may be used for determining authenticity of information.
- The **Certificate payload** transfers a public-key certificate.
- The Certificate Encoding field indicates the type of certificate or certificate-related information, which may include the following:

  • PKCS #7 wrapped X.509 certificate

  • PGP certificate

  • DNS signed key

  • X.509 certificate—signature

- X.509 certificate—key exchange
- Kerberos tokens
- Certificate Revocation List (CRL)
- Authority Revocation List (ARL)
- SPKI certificate

- At any point in an IKE exchange, the sender may include a **Certificate Request** payload to request the certificate of the other communicating entity.
- The **Authentication** payload contains data used for message authentication purposes.
- The **Nonce** payload contains random data used to guarantee liveness during an exchange and to protect against replay attacks.
- The **Notify** payload contains either error or status information associated with this SA or this SA negotiation.
- The following table lists the IKE notify messages.

Error Messages	Status Messages
Unsupported Critical Payload	Initial Contact
Invalid IKE SPI	Set Window Size
Invalid Major Version	Additional TS Possible
Invalid Syntax	IPCOMP Supported
Invalid Payload Type	NAT Detection Source IP
Invalid Message ID	NAT Detection Destination IP
Invalid SPI	Cookie
	Use Transport Mode

Error Messages	Status Messages
No Proposal Chosen	HTTP Cert Lookup Supported
Invalid KE Payload	Rekey SA
Authentication Failed	ESP TFC Padding Not Supported
Single Pair Required	Non First Fragments Also
No Additional SAS	
Internal Address Failure	
Failed CP Required	
TS Unacceptable	
Invalid Selectors	

- The **Delete** payload indicates one or more SAs that the sender has deleted from its database and that therefore are no longer valid.
- The **Vendor ID** payload contains a vendor-defined constant.
- The **Traffic Selector** payload allows peers to identify packet flows for processing by IPsec services.
- The **Encrypted** payload contains other payloads in encrypted form.
- The **Configuration** payload is used to exchange configuration information between IKE peers.
- The **Extensible Authentication Protocol (EAP)** payload allows IKE SAs to be authenticated using EAP.

Assessment questions to the lecture

Qn No	Question	Answer	Bloom's Knowledge Level
1	What is the primary function of IEEE 802.1X in a network? a) To encrypt data transmitted over the network b) To provide port-based authentication for devices connecting to the network	b	Remembering

	c) To establish a secure key exchange between devices d) To monitor and manage network traffic at Layer 3		
2	What is the main purpose of IPSec in networking? a) To authenticate users through port-based access control b) To provide secure communication through encryption and authentication c) To assign IP addresses to devices in the network d) To manage Quality of Service (QoS) policies in a network	b	Remembering
3	What is the role of Internet Key Exchange (IKE) in IPSec? a) To authenticate users connecting to the network b) To establish and manage Security Associations (SAs) for IPSec communication c) To encrypt the data packets transmitted over the network d) To control access to network resources based on device identity	b	Remembering

Students have to prepare answers for the following questions at the end of the lecture

Qn No	Question	Marks	CO	Bloom's Knowledge Level
1	Define Internet Key Exchange (IKE).	2	3	Remembering
2	Discuss IEEE 802.1X Port-Based Network Access Control in detail.	16	3	Understanding
3	Explain the working of IP Security (IPSec) and the role of Internet Key Exchange (IKE) in securing IP communication.	16	3	Understanding

Reference Book:

Author(s) Name	Title of the book	Page numbers
William Stallings	Cryptography and Network Security: Principles and Practice, 6th Edition	503-505,627-638,649-657

Unit	Access Control and Security	Lecture No	C313.3.3
Topic	Transport-Level Security: Web Security Considerations, Secure Sockets Layer, Transport Layer Security		

Learning Outcome (LO) At the end of this lecture, students will be able to		**Bloom's Knowledge Level**
LO1	Compare and contrast SSL and TLS.	Understanding
LO1	Discuss the working and security features of Secure Sockets Layer (SSL) and Transport Layer Security (TLS).	Understanding

Transport-Level Security:

Web Security Considerations

- The World Wide Web is fundamentally a client/server application running over the Internet and TCP/IP intranets.
- The following characteristics of Web usage suggest the need for tailored security tools:
  - Although Web browsers are very easy to use, Web servers are relatively easy to configure and manage, and Web content is increasingly easy to develop, the underlying software is extraordinarily complex.
  - A Web server can be exploited as a launching pad into the corporation's or agency's entire computer complex.
  - Casual and untrained (in security matters) users are common clients for Web based services.

Web Security Threats

- Table below provides a summary of the types of security threats faced when using the Web.

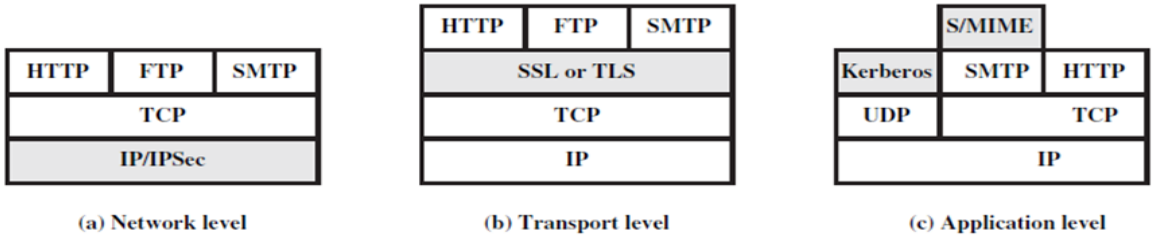
A Comparison of Threats on the Web

	Threats	Consequences	Countermeasures
Integrity	• Modification of user data • Trojan horse browser • Modification of memory • Modification of message traffic in transit	• Loss of information • Compromise of machine • Vulnerability to all other threats	Cryptographic checksums
Confidentiality	• Eavesdropping on the net • Theft of info from server • Theft of data from client • Info about network configuration • Info about which client talks to server	• Loss of information • Loss of privacy	Encryption, Web proxies
Denial of Service	• Killing of user threads • Flooding machine with bogus requests • Filling up disk or memory • Isolating machine by DNS attacks	• Disruptive • Annoying • Prevent user from getting work done	Difficult to prevent
Authentication	• Impersonation of legitimate users • Data forgery	• Misrepresentation of user • Belief that false information is valid	Cryptographic techniques

- One way to group these threats is in terms of passive and active attacks.
- Passive attacks include eavesdropping on network traffic between browser and server and gaining access to information on a Web site that is supposed to be restricted.
- Active attacks include impersonating another user, altering messages in transit between client and server, and altering information on a Web site.
- Another way to classify Web security threats is in terms of the location of the threat: Web server, Web browser, and network traffic between browser and server.

Web Traffic Security Approaches

- A number of approaches to providing Web security are possible.
- The various approaches that have been considered are similar in the services they provide and, to some extent, in the mechanisms that they use, but they differ with respect to their scope of applicability and their relative location within the TCP/IP protocol stack.
- Figure below illustrates this difference.



Relative Location of Security Facilities in the TCP/IP Protocol Stack

- One way to provide Web security is to use IP security (IPsec) (Figure 17.1a).
- The advantage of using IPsec is that it is transparent to end users and applications and provides a general-purpose solution.
- Furthermore, IPsec includes a filtering capability so that only selected traffic need incur the overhead of IPsec processing.
- Another relatively general-purpose solution is to implement security just above TCP (Figure b).
- The foremost example of this approach is the Secure Sockets Layer (SSL) and the follow-on Internet standard known as Transport Layer Security (TLS).
- At this level, there are two implementation choices.
- For full generality, SSL (or TLS) could be provided as part of the underlying protocol suite and therefore be transparent to applications.
- Alternatively, SSL can be embedded in specific packages.
- For example, Netscape and Microsoft Explorer browsers come equipped with SSL, and most Web servers have implemented the protocol.
- Application-specific security services are embedded within the particular application. Figure c shows examples of this architecture
- . The advantage of this approach is that the service can be tailored to the specific needs of a given application.

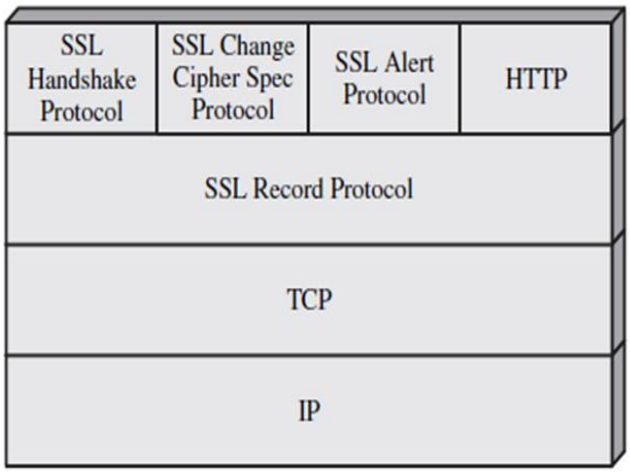
Secure Sockets Layer

- One of the most widely used security services is the Secure Sockets Layer (SSL) and the follow-on Internet standard known as Transport Layer Security (TLS), the latter defined in RFC 5246.
- SSL is a general-purpose service implemented as a set of protocols that rely on TCP.

- At this level, there are two implementation choices.
- For full generality, SSL (or TLS) could be provided as part of the underlying protocol suite and therefore be transparent to applications.
- Alternatively, SSL can be embedded in specific packages.
- For example, most browsers come equipped with SSL, and most Web servers have implemented the protocol.

SSL Architecture

- SSL is designed to make use of TCP to provide a reliable end-to-end secure service. SSL is not a single protocol but rather two layers of protocols, as illustrated in Figure below.



SSL Protocol Stack

- The SSL Record Protocol provides basic security services to various higher layer protocols.
- In particular, the Hypertext Transfer Protocol (HTTP), which provides the transfer service for Web client/server interaction, can operate on top of SSL.
- Three higher-layer protocols are defined as part of SSL: the Handshake Protocol, The Change Cipher Spec Protocol, and the Alert Protocol.
- Two important SSL concepts are the SSL session and the SSL connection, which are defined in the specification as follows.

  • **Connection:** A connection is a transport (in the OSI layering model definition) that provides a suitable type of service. For SSL, such connections are peer-to-peer relationships. The connections are transient. Every connection is associated with one session.

  • **Session:** An SSL session is an association between a client and a server. Sessions are created by the Handshake Protocol. Sessions define a set of cryptographic security parameters which can be shared among multiple connections. Sessions are used to avoid the expensive negotiation of new security parameters for each connection.

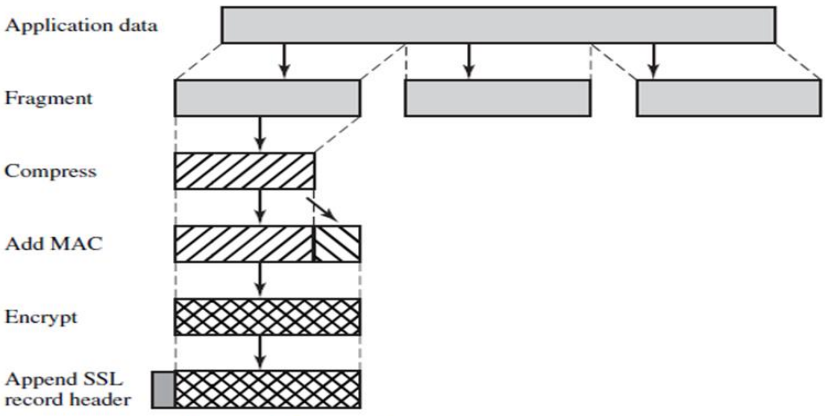
- There are a number of states associated with each session.
- Once a session is established, there is a current operating state for both read and write (i.e., receive and send).
- In addition, during the Handshake Protocol, pending read and write states are created.
- Upon successful conclusion of the Handshake Protocol, the pending states become the

current states.

- A session state is defined by the following parameters.
  • **Session identifier:** An arbitrary byte sequence chosen by the server to identify an active or resumable session state.
  • **Peer certificate:** An X509.v3 certificate of the peer. This element of the state may be null.
  • **Compression method:** The algorithm used to compress data prior to encryption.
  • **Cipher spec:** Specifies the bulk data encryption algorithm (such as null, AES, etc.) and a hash algorithm (such as MD5 or SHA-1) used for MAC calculation. It also defines cryptographic attributes such as the hash_size.
  • **Master secret:** 48-byte secret shared between the client and the server.
  • **Is resumable:** A flag indicating whether the session can be used to initiate new connections.
  A connection state is defined by the following parameters.
  • **Server and client random:** Byte sequences that are chosen by the server and client for each connection.
  • **Server write MAC secret:** The secret key used in MAC operations on data sent by the server.
  • **Client write MAC secret:** The secret key used in MAC operations on data sent by the client.
  • **Server write key:** The secret encryption key for data encrypted by the server and decrypted by the client.
  • **Client write key:** The symmetric encryption key for data encrypted by the client and decrypted by the server.
  • **Initialization vectors:** When a block cipher in CBC mode is used, an initialization vector (IV) is maintained for each key. This field is first initialized by the SSL Handshake Protocol. Thereafter, the final ciphertext block from each record is preserved for use as the IV with the following record.
  • **Sequence numbers:** Each party maintains separate sequence numbers for transmitted and received messages for each connection. When a party sends or receives a change cipher spec message, the appropriate sequence number is set to zero. Sequence numbers may not exceed $2^{64} - 1$.

SSL Record Protocol

- The SSL Record Protocol provides two services for SSL connections:
  • **Confidentiality:** The Handshake Protocol defines a shared secret key that is used for conventional encryption of SSL payloads.
  • **Message Integrity:** The Handshake Protocol also defines a shared secret key that is used to form a message authentication code (MAC).
- Figure below indicates the overall operation of the SSL Record Protocol.
- The Record Protocol takes an application message to be transmitted, fragments the data into manageable blocks, optionally compresses the data, applies a MAC, encrypts, adds a header, and transmits the resulting unit in a TCP segment. Received data are decrypted, verified, decompressed, and reassembled before being delivered to higher-level users.



SSL Record Protocol Operation

- The first step is **fragmentation**. Each upper-layer message is fragmented into blocks of 2^{14} bytes (16384 bytes) or less.
- Next, **compression** is optionally applied. Compression must be lossless and may not increase the content length by more than 1024 bytes.1
- In SSLv3 (as well as the current version of TLS), no compression algorithm is specified, so the default compression algorithm is null.
- The next step in processing is to compute a **message authentication code** over the compressed data.
- For this purpose, a shared secret key is used. The calculation is defined as

```
hash(MAC_write_secret || pad_2 ||
    hash(MAC_write_secret || pad_1 || seq_num ||
    SSLCompressed.type || SSLCompressed.length ||
    SSLCompressed.fragment))
```

where

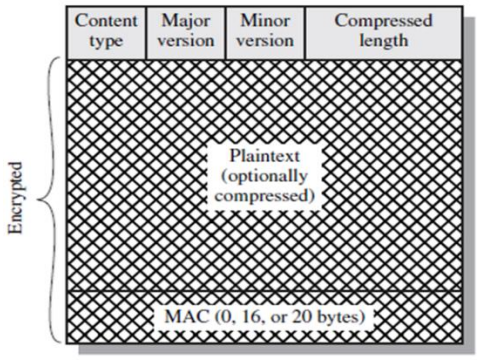
||	= concatenation
MAC_write_secret	= shared secret key
hash	= cryptographic hash algorithm; either MD5 or SHA-1
pad_1	= the byte 0x36 (0011 0110) repeated 48 times (384 bits) for MD5 and 40 times (320 bits) for SHA-1
pad_2	= the byte 0x5C (0101 1100) repeated 48 times for MD5 and 40 times for SHA-1
seq_num	= the sequence number for this message
SSLCompressed.type	= the higher-level protocol used to process this fragment
SSLCompressed.length	= the length of the compressed fragment
SSLCompressed.fragment	= the compressed fragment (if compression is not used, this is the plaintext fragment)

- The difference is that the two pads are concatenated in SSLv3 and are XORed in HMAC.
- The SSLv3 MAC algorithm is based on the original Internet draft for HMAC, which used concatenation.
- The final version of HMAC (defined in RFC 2104) uses the XOR.
- Next, the compressed message plus the MAC are **encrypted** using symmetric encryption.
- Encryption may not increase the content length by more than 1024 bytes, so that the total length may not exceed $2^{14} + 2048$.
- The following encryption algorithms are permitted:

Block Cipher		Stream Cipher	
Algorithm	**Key Size**	**Algorithm**	**Key Size**
AES	128, 256	RC4-40	40
IDEA	128	RC4-128	128
RC2-40	40		
DES-40	40		
DES	56		
3DES	168		
Fortezza	80		

- Fortezza can be used in a smart card encryption scheme.
- For stream encryption, the compressed message plus the MAC are encrypted.
- For block encryption, padding may be added after the MAC prior to encryption.
- The final step of SSL Record Protocol processing is to prepare a header consisting of the following fields:
  - **Content Type (8 bits):** The higher-layer protocol used to process the enclosed fragment.
  - **Major Version (8 bits):** Indicates major version of SSL in use. For SSLv3, the value is 3.
  - **Minor Version (8 bits):** Indicates minor version in use. For SSLv3, the value is 0.
  - **Compressed Length (16 bits):** The length in bytes of the plaintext fragment (or compressed fragment if compression is used). The maximum value is $2^{14} + 2048$.
- The content types that have been defined are change_cipher_spec, alert, handshake, and application_data.

- Figure below illustrates the SSL record format.



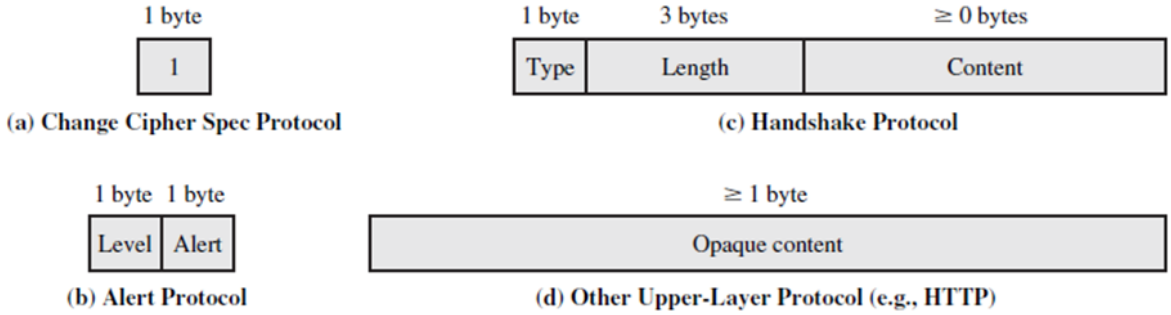
SSL Record Format

Change Cipher Spec Protocol

- The Change Cipher Spec Protocol is one of the three SSL-specific protocols that use the SSL Record Protocol, and it is the simplest.
- This protocol consists of a single message (Figure a), which consists of a single byte with the value 1.
- The sole purpose of this message is to cause the pending state to be copied into the current state, which updates the cipher suite to be used on this connection.

Alert Protocol

- The Alert Protocol is used to convey SSL-related alerts to the peer entity.
- As with other applications that use SSL, alert messages are compressed and encrypted, as specified by the current state.



SSL Record Protocol Payload

- Each message in this protocol consists of two bytes.
- The first byte takes the value warning (1) or fatal (2) to convey the severity of the message.
- If the level is fatal, SSL immediately terminates the connection.
- Other connections on the same session may continue, but no new connections on this session may be established.
- The second byte contains a code that indicates the specific alert.
- First, we list those alerts that are always fatal (definitions from the SSL specification):
  - unexpected_message: An inappropriate message was received.
  - bad_record_mac: An incorrect MAC was received.
  - decompression_failure: The decompression function received improper input (e.g., unable to decompress or decompress to greater than maximum allowable length).

• handshake_failure: Sender was unable to negotiate an acceptable set of security parameters given the options available.
• illegal_parameter: A field in a handshake message was out of range or inconsistent with other fields.
The remaining alerts are the following.
• close_notify: Notifies the recipient that the sender will not send any more messages on this connection. Each party is required to send a close_notify alert before closing the write side of a connection.
• no_certificate: May be sent in response to a certificate request if no appropriate certificate is available.
• bad_certificate: A received certificate was corrupt (e.g., contained a signature that did not verify).
• unsupported_certificate: The type of the received certificate is not supported.
• certificate_revoked: A certificate has been revoked by its signer.
• certificate_expired: A certificate has expired.
• certificate_unknown: Some other unspecified issue arose in processing the certificate, rendering it unacceptable.

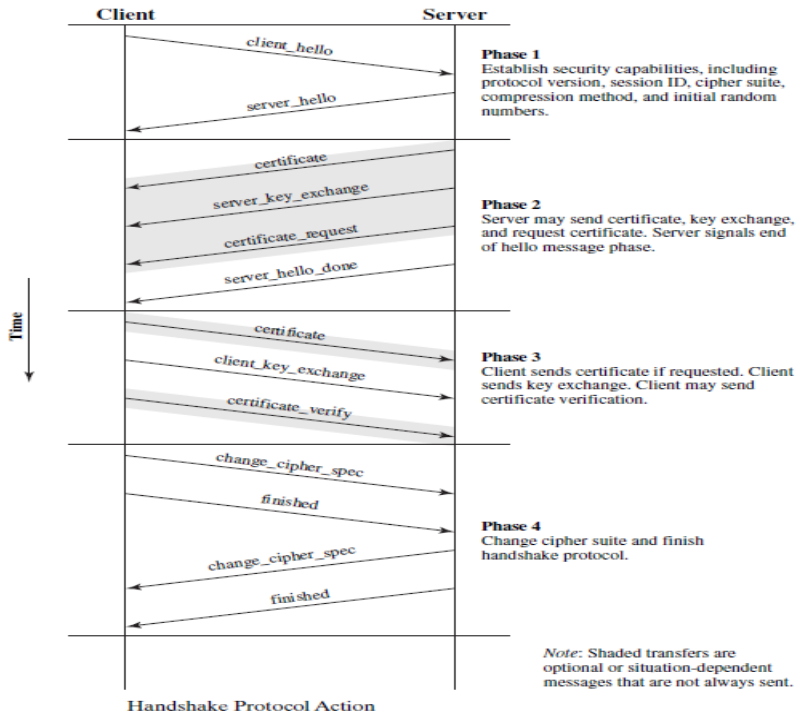
Handshake Protocol

- The most complex part of SSL is the Handshake Protocol.
- This protocol allows the server and client to authenticate each other and to negotiate an encryption and MAC algorithm and cryptographic keys to be used to protect data sent in an SSL record.
- The Handshake Protocol is used before any application data is transmitted.
- The Handshake Protocol consists of a series of messages exchanged by client and server. All of these have the format shown in Figure c.
- Each message has three fields:
  - **Type (1 byte):** Indicates one of ten messages. Table below lists the defined message types.
  - **Length (3 bytes):** The length of the message in bytes.
  - **Content (# 0 bytes):** The parameters associated with this message;

SSL Handshake Protocol Message Types

Message Type	Parameters
hello_request	null
client_hello	version, random, session id, cipher suite, compression method
server_hello	version, random, session id, cipher suite, compression method
certificate	chain of X.509v3 certificates
server_key_exchange	parameters, signature
certificate_request	type, authorities
server_done	null
certificate_verify	signature
client_key_exchange	parameters, signature
finished	hash value

- Figure below shows the initial exchange needed to establish a logical connection between client and server.



Handshake Protocol Action

- The exchange can be viewed as having four phases.

Phase 1. Establish Security Capabilities

- This phase is used to initiate a logical connection and to establish the security capabilities that will be associated with it.
- The exchange is initiated by the client, which sends a **client_hello message** with the following parameters:
  - **Version:** The highest SSL version understood by the client.
  - **Random:** A client-generated random structure consisting of a 32-bit timestamp and 28 bytes generated by a secure random number generator. These values serve as nonces and are used during key exchange to prevent replay attacks.
  - **Session ID:** A variable-length session identifier. A nonzero value indicates that the client wishes to update the parameters of an existing connection or to create a new connection on this session. A zero value indicates that the client wishes to establish a new connection on a new session.
  - **CipherSuite:** This is a list that contains the combinations of cryptographic algorithms supported by the client, in decreasing order of preference. Each element of the list (each cipher suite) defines both a key exchange algorithm and a CipherSpec; these are discussed subsequently.
  - **Compression Method:** This is a list of the compression methods the client supports.
- After sending the client_hello message, the client waits for the server_hello message, which contains the same parameters as the client_hello message.
- For the server_hello message, the following conventions apply.
- The Version field contains the lower of the versions suggested by the client and the highest supported by the server.
- The Random field is generated by the server and is independent of the client's Random field.

- If the SessionID field of the client was nonzero, the same value is used by the server; otherwise, the server's SessionID field contains the value for a new session.
- The CipherSuite field contains the single cipher suite selected by the server from those proposed by the client.
- The Compression field contains the compression method selected by the server from those proposed by the client.
- The first element of the CipherSuite parameter is the key exchange method (i.e., the means by which the cryptographic keys for conventional encryption and MAC are exchanged).
- The following key exchange methods are supported.
  - **Rsa:** The secret key is encrypted with the receiver's RSA public key. A public-key certificate for the receiver's key must be made available.
  - **Fixed Diffie-Hellman:** This is a Diffie-Hellman key exchange in which the server's certificate contains the Diffie-Hellman public parameters signed by the certificate authority (CA).
  - **Ephemeral Diffie-Hellman:** This technique is used to create ephemeral (temporary, one-time) secret keys. In this case, the Diffie-Hellman public keys are exchanged, signed using the sender's private RSA or DSS key. The receiver can use the corresponding public key to verify the signature. Certificates are used to authenticate the public keys. This would appear to be the most secure of the three Diffie-Hellman options, because it results in a temporary, authenticated key.
  - **Anonymous Diffie-Hellman:** The base Diffie-Hellman algorithm is used with no authentication. That is, each side sends its public Diffie-Hellman parameters to the other with no authentication. This approach is vulnerable to man-in-the-middle attacks, in which the attacker conducts anonymous Diffie- Hellman with both parties.
  - **Fortezza:** The technique defined for the Fortezza scheme.
- Following the definition of a key exchange method is the CipherSpec, which includes the following fields.
  - **CipherAlgorithm:** Any of the algorithms mentioned earlier: RC4, RC2, DES,3DES, DES40, IDEA, or Fortezza
  - **MACAlgorithm:** MD5 or SHA-1
  - **CipherType:** Stream or Block
  - **IsExportable:** True or False
  - **HashSize:** 0, 16 (for MD5), or 20 (for SHA-1) bytes
  - **Key Material:** A sequence of bytes that contain data used in generating the write keys
  - **IV Size:** The size of the Initialization Value for Cipher Block Chaining (CBC) encryption

Phase 2. Server Authentication and Key Exchange

- The server begins this phase by sending its certificate if it needs to be authenticated; the message contains one or a chain of X.509 certificates.
- The **certificate message** is required for any agreed-on key exchange method except anonymous Diffie-Hellman.
- Next, a **server_key_exchange message** may be sent if it is required.
- It is not required in two instances:
  (1) The server has sent a certificate with fixed Diffie- Hellman parameters or

(2) a RSA key exchange is to be used.

- The server_key_exchange message is needed for the following:
  - **Anonymous Diffie-Hellman:** The message content consists of the two global Diffie-Hellman values.
  - **Ephemeral Diffie-Hellman:** The message content includes the three Diffie-Hellman parameters provided for anonymous Diffie-Hellman plus a signature of those parameters.
  - **RSA key exchange (in which the server is using RSA but has a signature-only RSA key):** The server must create a temporary RSA public/private key pair and use the server_key_exchange message to send the public key. The message content includes the two parameters of the temporary RSA public key plus a signature of those parameters.
  - **Fortezza**

    Some further details about the signatures are warranted. As usual, a signature is created by taking the hash of a message and encrypting it with the sender's private key. In this case, the hash is defined as

    ```
    hash(ClientHello.random || ServerHello.random ||
    ServerParams)
    ```

- So the hash covers not only the Diffie-Hellman or RSA parameters but also the two nonces from the initial hello messages.
- This ensures against replay attacks and misrepresentation.
- Next, a nonanonymous server (server not using anonymous Diffie-Hellman) can request a certificate from the client.
- The **certificate_request message** includes two parameters: certificate_type and certificate_authorities.
- The certificate type indicates the public-key algorithm and its use:
  - RSA, signature only
  - DSS, signature only
  - RSA for fixed Diffie-Hellman; in this case the signature is used only for authentication,

    by sending a certificate signed with RSA
  - DSS for fixed Diffie-Hellman; again, used only for authentication
  - RSA for ephemeral Diffie-Hellman
  - DSS for ephemeral Diffie-Hellman
  - Fortezza
- The second parameter in the certificate_request message is a list of the distinguished names of acceptable certificate authorities.
- The final message in phase 2, and one that is always required, is the **server_done message**, which is sent by the server to indicate the end of the server hello and associated messages.
- After sending this message, the server will wait for a client response. This message has no parameters.

Phase 3. Client Authentication and Key Exchange

- Upon receipt of the server_done message, the client should verify that the server provided a valid certificate (if required) and check that the server_hello parameters are acceptable.
- If all is satisfactory, the client sends one or more messages back to the server.
- If the server has requested a certificate, the client begins this phase by sending a

certificate message.

- If no suitable certificate is available, the client sends a no_certificate alert instead.
- Next is the **client_key_exchange message**, which must be sent in this phase. The content of the message depends on the type of key exchange, as follows.
  - **Rsa:** The client generates a 48-byte *pre-master secret* and encrypts with the public key from the server's certificate or temporary RSA key from a server_key_exchange message. Its use to compute a *master secret* is explained later.
  - **Ephemeral or Anonymous Diffie-Hellman:** The client's public Diffie-Hellman parameters are sent.
  - **Fixed Diffie-Hellman:** The client's public Diffie-Hellman parameters were sent in a certificate message, so the content of this message is null.
  - **Fortezza:** The client's Fortezza parameters are sent.
- Finally, in this phase, the client may send a **certificate_verify message** to provide explicit verification of a client certificate.
- This message is only sent following any client certificate that has signing capability (i.e., all certificates except those containing fixed Diffie-Hellman parameters).
- This message signs a hash code based on the preceding messages, defined as

```
CertificateVerify.signature.md5_hash=
     MD5(master_secret || pad_2 || MD5(handshake_messages ||
         master_secret || pad_1));
CertificateVerify.signature.sha_hash=
     SHA(master_secret || pad_2 || SHA(handshake_messages ||
         master_secret || pad_1));
```

where pad_1 and pad_2 are the values defined earlier for the MAC, **handshake_ messages** refers to all Handshake Protocol messages sent or received starting at client_hello but not including this message.

- If the user's private key is DSS, then it is used to encrypt the SHA-1 hash. If the user's private key is RSA, it is used to encrypt the concatenation of the MD5 and SHA-1 hashes. In either case, the purpose is to verify the client's ownership of the private key for the client certificate. Even if someone is misusing the client's certificate, he or she would be unable to send this message.

Phase 4. Finish

- This phase completes the setting up of a secure connection.
- The client sends a change_cipher_spec message and copies the pending CipherSpec into the current CipherSpec.
- The client then immediately sends the **finished message** under the new algorithms, keys, and secrets.
- The finished message verifies that the key exchange and authentication processes were successful.
- The content of the finished message is the concatenation of two hash values:

```
MD5(master_secret || pad2 || MD5(handshake_messages ||
    Sender || master_secret || pad1))
SHA(master_secret || pad2 || SHA(handshake_messages ||
    Sender || master_secret || pad1))
```

where Sender is a code that identifies that the sender is the client and handshake_messages is all of the data from all handshake messages up to but not including this message.

- In response to these two messages, the server sends its own change_ cipher_spec

message, transfers the pending to the current CipherSpec, and sends its finished message.

- At this point, the handshake is complete and the client and server may begin to exchange application-layer data.

Cryptographic Computations

Two further items are of interest:
(1) the creation of a shared master secret by means of the key exchange and
(2) the generation of cryptographic parameters from the master secret.

Master Secret Creation

- The shared master secret is a one-time 48-byte value (384 bits) generated for this session by means of secure key exchange.
- The creation is in two stages.
- First, a pre_master_secret is exchanged.
- Second, the master_ secret is calculated by both parties. For pre_master_secret exchange, there are two possibilities.
  - **RSA**: A 48-byte pre_master_secret is generated by the client, encrypted with the server's public RSA key, and sent to the server. The server decrypts the ciphertext using its private key to recover the pre_master_secret.
  - **Diffie-Hellman:** Both client and server generate a Diffie-Hellman public key.After these are exchanged, each side performs the Diffie-Hellman calculation to create the shared pre_master_secret.
- Both sides now compute the master_secret as

```
master_secret = MD5(pre_master_secret || SHA('A' ||
                    pre_master_secret || ClientHello.random ||
                    ServerHello.random)) ||
                MD5(pre_master_secret || SHA('BB' ||
                    pre_master_secret || ClientHello.random ||
                    ServerHello.random)) ||
                MD5(pre_master_secret || SHA('CCC' ||
                    pre_master_secret || ClientHello.random ||
                    ServerHello.random))
```

where ClientHello.random and ServerHello.random are the two nonce values exchanged in the initial hello messages.

Generation of Cryptographic Parameters

- CipherSpecs require a client write MAC secret, a server write MAC secret, a client write key, a server write key, a client write IV, and a server write IV, which are generated from the master secret in that order.
- These parameters are generated from the master secret by hashing the master secret into a sequence of secure bytes of sufficient length for all needed parameters.
- The generation of the key material from the master secret uses the same format for generation of the master secret from the pre-master secret as

```
key_block = MD5(master_secret || SHA('A' || master_secret ||
                ServerHello.random || ClientHello.random)) ||
            MD5(master_secret || SHA('BB' || master_secret ||
                ServerHello.random || ClientHello.random)) ||
            MD5(master_secret || SHA('CCC' || master_secret ||
                ServerHello.random || ClientHello.random)) || ...
```

until enough output has been generated.

- The result of this algorithmic structure is a pseudorandom function.
- The client and server random numbers can be viewed as salt values to complicate cryptanalysis.

Transport Layer Security

- TLS is an IETF standardization initiative whose goal is to produce an Internet standard version of SSL.
- TLS is defined as a Proposed Internet Standard in RFC 5246.
- RFC 5246 is very similar to SSLv3. In this section, we highlight the differences.

Version Number

- The TLS Record Format is the same as that of the SSL Record Format, and the fields in the header have the same meanings.
- The one difference is in version values.
- For the current version of TLS, the major version is 3 and the minor version is 3.

Message Authentication Code

- There are two differences between the SSLv3 and TLS MAC schemes: the actual algorithm and the scope of the MAC calculation.
- TLS makes use of the HMAC algorithm defined in RFC 2104. Recall from Chapter 12 that HMAC is defined as

$$\text{HMAC}_K(M) = \text{H}[(K^+ \oplus \text{opad}) \parallel \text{H}[(K^+ \oplus \text{ipad}) \parallel M]]$$

where

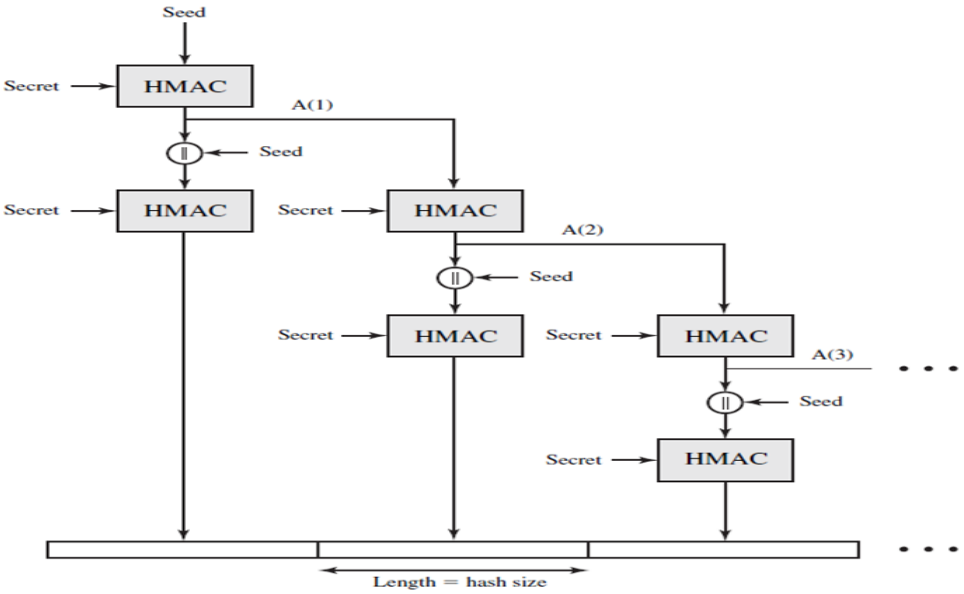
H = embedded hash function (for TLS, either MD5 or SHA-1)
M = message input to HMAC
K^+ = secret key padded with zeros on the left so that the result is equal to the block length of the hash code (for MD5 and SHA-1, block length = 512 bits)
ipad = 00110110 (36 in hexadecimal) repeated 64 times (512 bits)
opad = 01011100 (5C in hexadecimal) repeated 64 times (512 bits)

- SSLv3 uses the same algorithm, except that the padding bytes are concatenated with the secret key rather than being XORed with the secret key padded to the block length.
- The level of security should be about the same in both cases.
- For TLS, the MAC calculation encompasses the fields indicated in the following expression:
- The MAC calculation covers all of the fields covered by the SSLv3 calculation, plus the field TLSCompressed.version, which is the version of the protocol being employed.

```
MAC(MAC_write_secret, seq_num || TLSCompressed.type ||
    TLSCompressed.version || TLSCompressed.length ||
    TLSCompressed.fragment)
```

Pseudorandom Function

- TLS makes use of a pseudorandom function referred to as PRF to expand secrets into blocks of data for purposes of key generation or validation.
- The objective is to make use of a relatively small shared secret value but to generate longer blocks of data in a way that is secure from the kinds of attacks made on hash functions and MACs.



TLS Function P_hash(secret, seed)

- The PRF is based on the data expansion function given as

```
P_hash(secret, seed) = HMAC_hash(secret,A(1) || seed) ||
                       HMAC_hash(secret,A(2) || seed) ||
                       HMAC_hash(secret,A(3) || seed) || . . .
```

where A() is defined as

A(0) = seed
A(i) = HMAC_hash(secret, A(i − 1))

- The data expansion function makes use of the HMAC algorithm with either MD5 or SHA-1 as the underlying hash function.
- As can be seen, P_hash can be iterated as many times as necessary to produce the required quantity of data.
- To make PRF as secure as possible, it uses two hash algorithms in a way that should guarantee its security if either algorithm remains secure. PRF is defined as

```
PRF(secret, label, seed) = P_hash(S1, label || seed)
```

- PRF takes as input a secret value, an identifying label, and a seed value and produces an output of arbitrary length.

Alert Codes

- TLS supports all of the alert codes defined in SSLv3 with the exception of no_certificate. A number of additional codes are defined in TLS; of these, the following are always fatal.
  - **record_overflow:** A TLS record was received with a payload (ciphertext) whose length exceeds 214 + 2048 bytes, or the ciphertext decrypted to a length of greater than 2^{14} + 1024 bytes.
  - **unknown_ca:** A valid certificate chain or partial chain was received, but the certificate was not accepted because the CA certificate could not be located or could not be matched with a known, trusted CA.
  - **access_denied:** A valid certificate was received, but when access control was applied, the sender decided not to proceed with the negotiation.
  - **decode_error:** A message could not be decoded, because either a field was

out of its specified range or the length of the message was incorrect.

• **protocol_version:** The protocol version the client attempted to negotiate is recognized but not supported.

• **insufficient_security:** Returned instead of handshake_failure when a negotiation has failed specifically because the server requires ciphers more secure than those supported by the client.

• **unsupported_extension:** Sent by clients that receive an extended server hello containing an extension not in the corresponding client hello.

• **internal_error:** An internal error unrelated to the peer or the correctness of the protocol makes it impossible to continue.

• **decrypt_error:** A handshake cryptographic operation failed, including being unable to verify a signature, decrypt a key exchange, or validate a finished message.

The remaining alerts include the following.

• **user_canceled:** This handshake is being canceled for some reason unrelated to a protocol failure.

• **no_renegotiation:** Sent by a client in response to a hello request or by the server in response to a client hello after initial handshaking. Either of these messages would normally result in renegotiation, but this alert indicates that the sender is not able to renegotiate. This message is always a warning.

Cipher Suites

- There are several small differences between the cipher suites available under SSLv3 and under TLS:

• **Key Exchange:** TLS supports all of the key exchange techniques of SSLv3 with the exception of Fortezza.

• **Symmetric Encryption Algorithms:** TLS includes all of the symmetric encryption

algorithms found in SSLv3, with the exception of Fortezza.

Client Certificate Types

- TLS defines the following certificate types to be requested in a certificate_request message: rsa_sign, dss_sign, rsa_fixed_dh, and dss_fixed_dh.
- These are all defined in SSLv3.
- In addition, SSLv3 includes rsa_ephemeral_dh,dss_ephemeral_dh, and fortezza_kea. Ephemeral Diffie-Hellman involves signing the Diffie-Hellman parameters with either RSA or DSS.
- For TLS, the rsa_sign and dss_sign types are used for that function; a separate signing type is not needed to sign Diffie-Hellman parameters.
- TLS does not include the Fortezza scheme.

certificate_verify and Finished Messages

- In the TLS certificate_verify message, the MD5 and SHA-1 hashes are calculated only over handshake_messages.
- As with the finished message in SSLv3, the finished message in TLS is a hash based on the shared master_secret, the previous handshake messages, and a label that identifies client or server.
- The calculation is somewhat different.
- For TLS, we have

```
PRF(master_secret,finished_label,MD5(handshake_messages)||
    SHA-1(handshake_messages))
```

where finished_label is the string "client finished" for the client and "server finished" for the server.

Cryptographic Computations

- The pre_master_secret for TLS is calculated in the same way as in SSLv3. As in SSLv3, the master_secret in TLS is calculated as a hash function of the pre_master_secret and the two hello random numbers.
- The form of the TLS calculation is different from that of SSLv3 and is defined as

```
master_secret = PRF(pre_master_secret,"master secret",
                ClientHello.random||ServerHello.random)
```

- The algorithm is performed until 48 bytes of pseudorandom output are produced. The calculation of the key block material (MAC secret keys, session encryption keys, and IVs) is defined as

```
key_block = PRF(master_secret, "key expansion",
            SecurityParameters.server_random ||
            SecurityParameters.client_random)
```

until enough output has been generated.

- As with SSLv3, the key_block is a function of the master_secret and the client and server random numbers, but for TLS, the actual algorithm is different.

Padding

- In SSL, the padding added prior to encryption of user data is the minimum amount required so that the total size of the data to be encrypted is a multiple of the cipher's block length.
- In TLS, the padding can be any amount that results in a total that is a multiple of the cipher's block length, up to a maximum of 255 bytes.
- For example, if the plaintext (or compressed text if compression is used) plus MAC plus padding. length byte is 79 bytes long, then the padding length (in bytes) can be 1, 9, 17, and so on, up to 249.
- A variable padding length may be used to frustrate attacks based on an analysis of the lengths of exchanged messages.

Assessment questions to the lecture

Qn No	Question	Answer	Bloom's Knowledge Level
1	What is the primary purpose of SSL/TLS in web security? a)Data compression b) Encrypting data for secure transmission c) Increasing data transfer speed d) Authenticating hardware devices	b	Remembering

2	Which port is commonly used for HTTPS communication secured with SSL/TLS? a) 21 b) 25 c) 80 d) 443	d	Remembering
3	Which protocol is typically replaced by TLS in modern secure web communications? a) HTTP b) FTP c) SSL d) SMTP	c	Remembering

Students have to prepare answers for the following questions at the end of the lecture

Qn No	Question	Marks	CO	Bloom's Knowledge Level
1	What are the key differences between SSL and TLS?	2	3	Remembering
2	Compare and contrast SSL and TLS. Discuss their use in securing communications over the internet.	8	3	Understanding
3	Discuss the working and security features of Secure Sockets Layer (SSL) and Transport Layer Security (TLS). Explain how they ensure confidentiality and integrity of data.	16	3	Understanding

Reference Book:

Author(s) Name	Title of the book	Page numbers
William Stallings	Cryptography and Network Security: Principles and Practice, 6th Edition	523-543

Unit	Access Control and Security	Lecture No	C313.3.4
Topic	HTTPS standard, Secure Shell (SSH) application		
Learning Outcome (LO) At the end of this lecture, students will be able to			**Bloom's Knowledge Level**
LO1	Define HTTPS and Secure Shell (SSH) and their applications		Remembering
LO2	Describe the architecture of HTTPS, including how it ensures secure communication between a client and server.		Understanding
LO3	Explain the SSH protocol stack		Understanding

HTTPS

- HTTPS (HTTP over SSL) refers to the combination of HTTP and SSL to implement secure communication between a Web browser and a Web server.
- The HTTPS capability is built into all modern Web browsers.
- Its use depends on the Web server supporting HTTPS communication.
- For example, some search engines do not support HTTPS. Google provides HTTPS as an option: https://google.com.
- The principal difference seen by a user of a Web browser is that URL (uniform resource locator) addresses begin with https:// rather than http://.
- A normal HTTP connection uses port 80. If HTTPS is specified, port 443 is used, which invokes SSL.
- When HTTPS is used, the following elements of the communication are encrypted:
  - URL of the requested document
  - Contents of the document
  - Contents of browser forms (filled in by browser user)
  - Cookies sent from browser to server and from server to browser
  - Contents of HTTP header
- HTTPS is documented in RFC 2818, *HTTP Over TLS*.
- There is no fundamental change in using HTTP over either SSL or TLS, and both implementations are referred to as HTTPS.

Connection Initiation

- For HTTPS, the agent acting as the HTTP client also acts as the TLS client.
- The client initiates a connection to the server on the appropriate port and then sends the TLS ClientHello to begin the TLS handshake.
- When the TLS handshake has finished, the client may then initiate the first HTTP request.
- All HTTP data is to be sent as TLS application data.
- Normal HTTP behavior, including retained connections, should be followed.
- There are three levels of awareness of a connection in HTTPS.
  - At the HTTP level, an HTTP client requests a connection to an HTTP server by sending a connection request to the next lowest layer.
  - Typically, the next lowest layer is TCP, but it also may be TLS/SSL.
  - At the level of TLS, a session is established between a TLS client and a TLS server. This session can support one or more connections at any time.
- A TLS request to establish a connection begins with the establishment of a TCP

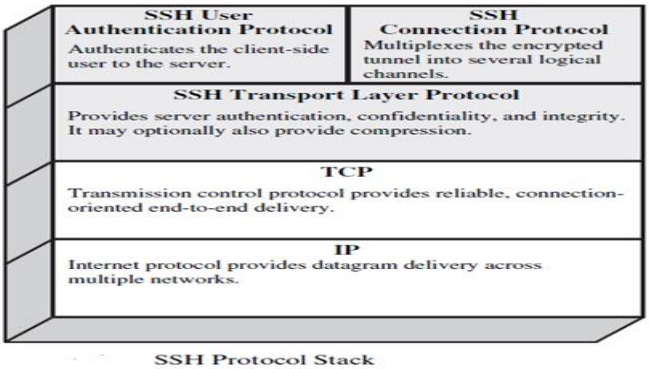
connection between the TCP entity on the client side and the TCP entity on the server side.

Connection Closure

- An HTTP client or server can indicate the closing of a connection by including the following line in an HTTP record: Connection: close.
- This indicates that the connection will be closed after this record is delivered.
- The closure of an HTTPS connection requires that TLS close the connection with the peer TLS entity on the remote side, which will involve closing the underlying TCP connection.
- At the TLS level, the proper way to close a connection is for each side to use the TLS alert protocol to send a close_notify alert.
- TLS implementations must initiate an exchange of closure alerts before closing a connection.
- A TLS implementation may, after sending a closure alert, close the connection without waiting for the peer to send its closure alert, generating an "incomplete close".
- HTTP clients also must be able to cope with a situation in which the underlying TCP connection is terminated without a prior close_notify alert and without a Connection: close indicator.
- Such a situation could be due to a programming error on the server or a communication error that causes the TCP connection to drop.
- However, the unannounced TCP closure could be evidence of some sort of attack.
- So the HTTPS client should issue some sort of security warning when this occurs.

Secure Shell (SSH)

- Secure Shell (SSH) is a protocol for secure network communications designed to be relatively simple and inexpensive to implement.
- The initial version, SSH1 was focused on providing a secure remote logon facility to replace TELNET and other remote logon schemes that provided no security.
- SSH also provides a more general client/server capability and can be used for such network functions as file transfer and e-mail.
- A new version, SSH2, fixes a number of security flaws in the original scheme.
- SSH2 is documented as a proposed standard in IETF RFCs 4250 through 4256.
- SSH client and server applications are widely available for most operating systems.
- It has become the method of choice for remote login and X tunneling and is rapidly becoming one of the most pervasive applications for encryption technology outside of embedded systems.
- SSH is organized as three protocols that typically run on top of TCP:



SSH Protocol Stack

• **Transport Layer Protocol:** Provides server authentication, data confidentiality, and data integrity with forward secrecy (i.e., if a key is compromised during one session, the knowledge does not affect the security of earlier sessions).The transport layer may optionally provide compression.
• **User Authentication Protocol:** Authenticates the user to the server.
• **Connection Protocol:** Multiplexes multiple logical communications channels over a single, underlying SSH connection.

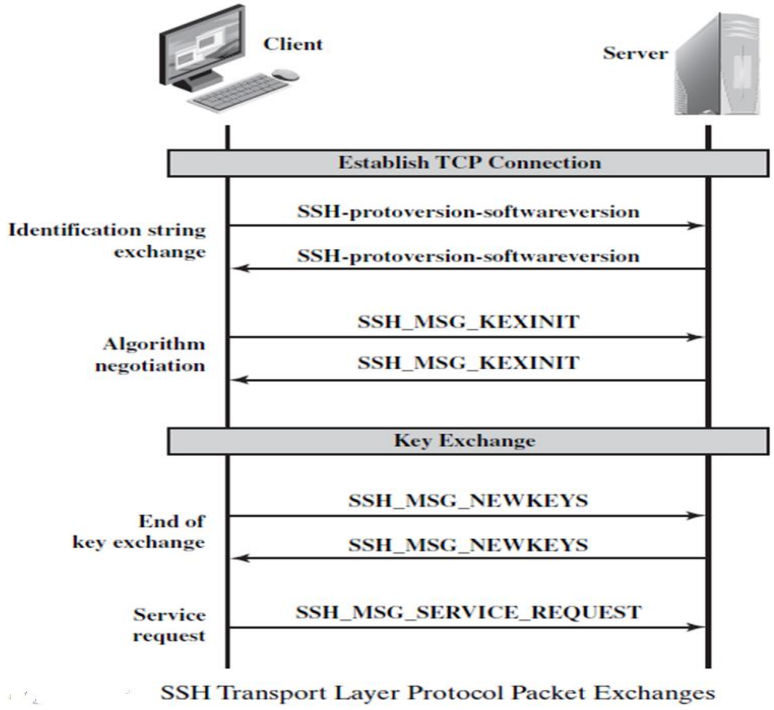
Transport Layer Protocol

Host Keys

- Server authentication occurs at the transport layer, based on the server possessing a public/private key pair.
- A server may have multiple host keys using multiple different asymmetric encryption algorithms.
- Multiple hosts may share the same host key.
- In any case, the server host key is used during key exchange to authenticate the identity of the host.
- For this to be possible, the client must have a priori knowledge of the server's public host key. RFC 4251 dictates two alternativetrust models that can be used:
    1. The client has a local database that associates each host name (as typed by the user) with the corresponding public host key.
    2. The host name-to-key association is certified by a trusted certification authority (CA).

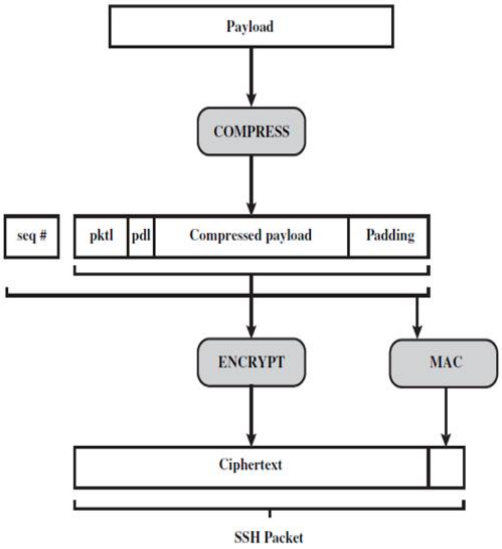
Packet Exchange

- Figure below illustrates the sequence of events in the SSH Transport Layer Protocol. First, the client establishes a TCP connection to the server.



SSH Transport Layer Protocol Packet Exchanges

- This is done via the TCP protocol and is not part of the Transport Layer Protocol. Once the connection is established, the client and server exchange data, referred to as packets, in the data field of a TCP segment.
- Each packet is in the following format



SSH Transport Layer Protocol Packet Formation

• **Packet length:** Length of the packet in bytes, not including the packet length and MAC fields.
• **Padding length:** Length of the random padding field.
• **Payload:** Useful contents of the packet. Prior to algorithm negotiation, this field is uncompressed. If compression is negotiated, then in subsequent packets, this field is compressed.
• **Random padding:** Once an encryption algorithm has been negotiated, this field is added. It contains random bytes of padding so that that total length of the packet (excluding the MAC field) is a multiple of the cipher block size, or 8 bytes for a stream cipher.
• **Message authentication code (MAC):** If message authentication has been negotiated, this field contains the MAC value.

- The SSH Transport Layer packet exchange consists of a sequence of steps
- The first step, the **identification string exchange**, begins with the client sending a packet with an identification string of the form:

```
SSH-protoversion-softwareversion SP comments CR LF
```

where SP, CR, and LF are space character, carriage return, and line feed, respectively.

- Table below shows the allowable options for encryption, MAC, and compression.

SSH Transport Layer Cryptographic Algorithms

Cipher	
3des-cbc*	Three-key 3DES in CBC mode
blowfish-cbc	Blowfish in CBC mode
twofish256-cbc	Twofish in CBC mode with a 256-bit key
twofish192-cbc	Twofish with a 192-bit key
twofish128-cbc	Twofish with a 128-bit key
aes256-cbc	AES in CBC mode with a 256-bit key
aes192-cbc	AES with a 192-bit key
aes128-cbc**	AES with a 128-bit key
Serpent256-cbc	Serpent in CBC mode with a 256-bit key
Serpent192-cbc	Serpent with a 192-bit key
Serpent128-cbc	Serpent with a 128-bit key
arcfour	RC4 with a 128-bit key
cast128-cbc	CAST-128 in CBC mode

MAC algorithm	
hmac-sha1*	HMAC-SHA1; digest length = key length = 20
hmac-sha1-96**	First 96 bits of HMAC-SHA1; digest length = 12; key length = 20
hmac-md5	HMAC-MD5; digest length = key length = 16
hmac-md5-96	First 96 bits of HMAC-MD5; digest length = 12; key length = 16

Compression algorithm	
none*	No compression
zlib	Defined in RFC 1950 and RFC 1951

* = Required
** = Recommended

- For each category, the algorithm chosen is the first algorithm on the client's list that is also supported by the server.
- The next step is **key exchange**. The specification allows for alternative methods of key exchange, but at present, only two versions of Diffie-Hellman key exchange are specified.
- Both versions are defined in RFC 2409 and require only one packet in each direction.
- The following steps are involved in the exchange.
- In this, C is the client; S is the server; p is a large safe prime; g is a generator for a subgroup of GF(p); q is the order of the subgroup; V_S is S's identification string; V_C is C's identification string; K_S is S's public host key; I_C is C's SSH_MSG_KEXINIT message and I_S is S's SSH_MSG_KEXINIT message that have been exchanged before this part begins.
- The values of p, g, and q are known to both client and server as a result of the algorithm selection negotiation.
- The hash function hash() is also decided during algorithm negotiation.
    1. C generates a random number $x(1 < x < q)$ and computes $e = g^x \bmod p$. C sends e to S.
    2. S generates a random number $y(0 < y < q)$ and computes $f = g^y \bmod p$. S receives e. It computes $K = e^y \bmod p$, $H = \text{hash}(\text{V\_C} \| \text{V\_S} \| \text{I\_C} \| \text{I\_S} \| \text{K\_S} \| e \| f \| \text{K})$, and signature s on H with its private host key. S sends $(\text{K\_S} \| f \| s)$ to C. The signing operation may involve a second hashing operation.
    3. C verifies that K_S really is the host key for S (e.g., using certificates or a local database). C is also allowed to accept the key without verification; however, doing so will render the protocol insecure against active attacks (but may be desirable for practical reasons in the short term in many environments). C then computes $K = f^x \bmod p$, $H = \text{hash}(\text{V\_C} \| \text{V\_S} \| \text{I\_C} \| \text{I\_S} \| \text{K\_S} \| e \| f \| K)$, and verifies the signature s on H.
- As a result of these steps, the two sides now share a master key K.
- In addition, the server has been authenticated to the client, because the server has used its private key to sign its half of the Diffie-Hellman exchange.
- Finally, the hash value H serves as a session identifier for this connection.
- Once computed, the session identifier is not changed, even if the key exchange is

performed again for this connection to obtain fresh keys.

- The **end of key exchange** is signaled by the exchange of SSH_MSG_NEWKEYS packets.
- At this point, both sides may start using the keys generated from *K*.
- The final step is **service request**. The client sends an SSH_MSG_SERVICE_ REQUEST packet to request either the User Authentication or the Connection Protocol.
- Subsequent to this, all data is exchanged as the payload of an SSH Transport Layer packet, protected by encryption and MAC.

Key Generation

- The keys used for encryption and MAC (and any needed IVs) are generated from the shared secret key *K*, the hash value from the key exchange *H*, and the session identifier, which is equal to *H* unless there has been a subsequent key exchange after the initial key exchange.
- The values are computed as follows.
  - Initial IV client to server: HASH($K \| H \| \text{"A"} \| \text{session\_id}$)
  - Initial IV server to client: HASH($K \| H \| \text{"B"} \| \text{session\_id}$)
  - Encryption key client to server: HASH($K \| H \| \text{"C"} \| \text{session\_id}$)
  - Encryption key server to client: HASH($K \| H \| \text{"D"} \| \text{session\_id}$)
  - Integrity key client to server: HASH($K \| H \| \text{"E"} \| \text{session\_id}$)
  - Integrity key server to client: HASH($K \| H \| \text{"F"} \| \text{session\_id}$)

where HASH() is the hash function determined during algorithm negotiation.

User Authentication Protocol

- The User Authentication Protocol provides the means by which the client is authenticated to the server.

Message Types and Formats

- Three types of messages are always used in the User Authentication Protocol. Authentication requests from the client have the format:

byte	SSH_MSG_USERAUTH_REQUEST (50)
string	user name
string	service name
string	method name
…	method specific fields

If the server either (1) rejects the authentication request or (2) accepts the request but requires one or more additional authentication methods, the server sends a message with the format:

byte	SSH_MSG_USERAUTH_FAILURE (51)
name-list	authentications that can continue
boolean	partial success

where the name-list is a list of methods that may productively continue the dialog.

If the server accepts authentication, it sends a single byte message: SSH_MSG_USERAUTH_SUCCESS (52).

Message Exchange

The message exchange involves the following steps.

1. The client sends a SSH_MSG_USERAUTH_REQUEST with a requested method of none.
2. The server checks to determine if the user name is valid. If not, the server returns SSH_MSG_USERAUTH_FAILURE with the partial success value of false. If the user

name is valid, the server proceeds to step 3.

3. The server returns SSH_MSG_USERAUTH_FAILURE with a list of one or more authentication methods to be used.
4. The client selects one of the acceptable authentication methods and sends a SSH_MSG_USERAUTH_REQUEST with that method name and the required method-specific fields. At this point, there may be a sequence of exchanges to perform the method.
5. If the authentication succeeds and more authentication methods are required, the server proceeds to step 3, using a partial success value of true. If the authentication fails, the server proceeds to step 3, using a partial success value of false.
6. When all required authentication methods succeed, the server sends a SSH_MSG_USERAUTH_SUCCESS message, and the Authentication Protocol is over.

Authentication Methods

- The server may require one or more of the following authentication methods.
  - **publickey:** The details of this method depend on the public-key algorithm chosen.
  - **password:** The client sends a message containing a plaintext password, which is protected by encryption by the Transport Layer Protocol.
  - **hostbased:** Authentication is performed on the client's host rather than the client itself.

Connection Protocol

- The SSH Connection Protocol runs on top of the SSH Transport Layer Protocol and assumes that a secure authentication connection is in use.
- That secure authentication connection, referred to as a **tunnel,** is used by the Connection Protocol to multiplex a number of logical channels.

Channel Mechanism

- All types of communication using SSH, such as a terminal session, are supported using separate channels.
- Either side may open a channel.
- For each channel, each side associates a unique channel number, which need not be the same on both ends.
- Channels are flow controlled using a window mechanism.
- No data may be sent to a channel until a message is received to indicate that window space is available.
- The life of a channel progresses through three stages: opening a channel, data transfer, and closing a channel.
- When either side wishes to **open a new channel,** it allocates a local number for the channel and then sends a message of the form:

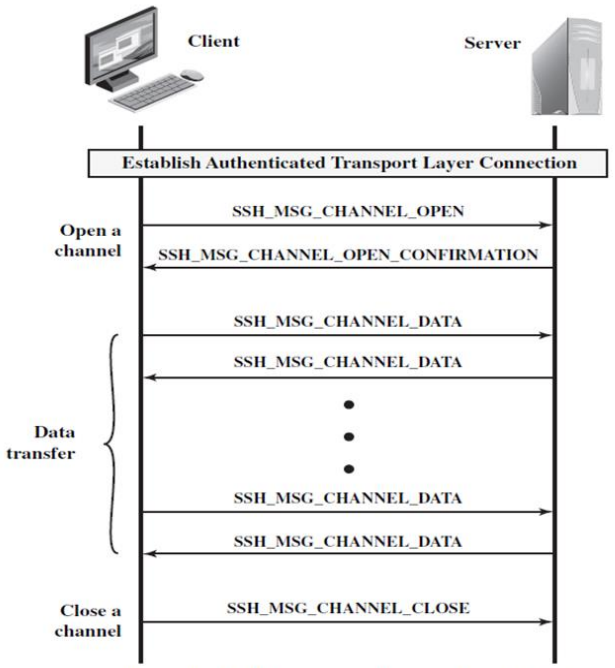
byte	SSH_MSG_CHANNEL_OPEN
string	channel type
uint32	sender channel
uint32	initial window size
uint32	maximum packet size
....	channel type specific data follows

where uint32 means unsigned 32-bit integer.

- If the remote side is able to open the channel, it returns a

SSH_MSG_CHANNEL_OPEN_CONFIRMATION message, which includes the sender channel number, the recipient channel number, and window and packet size values for incoming traffic.

- Otherwise, the remote side returns a SSH_MSG_CHANNEL_OPEN_FAILURE message with a reason code indicating the reason for failure.
- Once a channel is open, **data transfer** is performed using a SSH_MSG_CHANNEL_DATA message, which includes the recipient channel number and a block of data.
- These messages, in both directions, may continue as long as the channel is open.
- When either side wishes to **close a channel,** it sends a SSH_MSG_CHANNEL_CLOSE message, which includes the recipient channel number.
- Figure below provides an example of Connection Protocol Message Exchange.



Example of SSH Connection Protocol Message Exchange

Channel Types

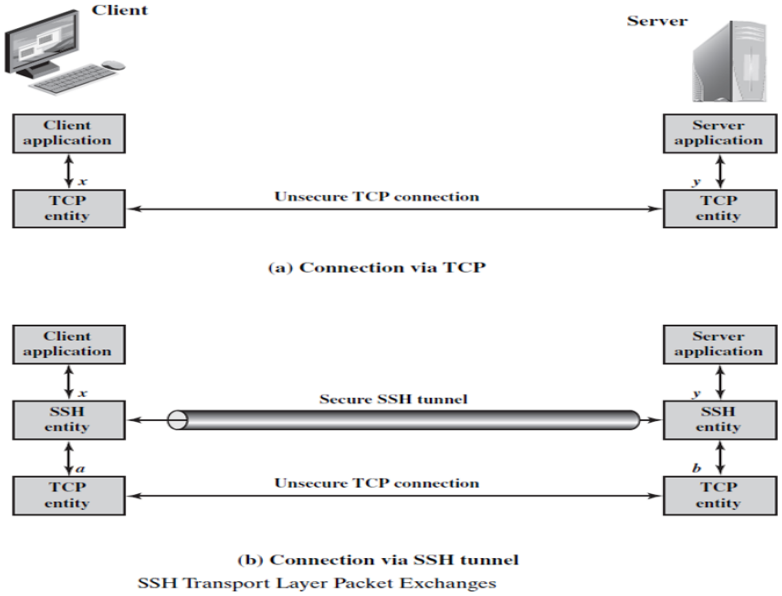
- Four channel types are recognized in the SSH Connection Protocol specification.
  • **session:** The remote execution of a program.
  • **x11:** This refers to the X Window System, a computer software system and network protocol that provides a graphical user interface (GUI) for networked computers.
  • **forwarded-tcpip:** This is remote port forwarding, as explained in the next subsection.
  • **direct-tcpip:** This is local port forwarding, as explained in the next subsection.

Port Forwarding

- One of the most useful features of SSH is port forwarding.
- In essence, port forwarding provides the ability to convert any insecure TCP connection into a secure SSH connection. This is also referred to as SSH tunneling.
- A **port** is an identifier of a user of TCP. So, any application that runs on top of TCP has a port number.
- Incoming TCP traffic is delivered to the appropriate application on the basis of the port

number.

- An application may employ multiple port numbers.
- For example, for the Simple Mail Transfer Protocol (smtp), the server side generally listens on port 25, so an incoming SMTP request uses TCP and addresses the data to destination port 25. TCP recognizes that this is the SMTP server address and routes the data to the SMTP server application.
- Figure below illustrates the basic concept behind port forwarding.



(a) Connection via TCP

(b) Connection via SSH tunnel

SSH Transport Layer Packet Exchanges

- We have a client application that is identified by port number *x* and a server application identified by port number *y*.
- At some point, the client application invokes the local TCP entity and requests a connection to the remote server on port *y*.
- The local TCP entity negotiates a TCP connection with the remote TCP entity, such that the connection links local port *x* to remote port *y*.
- To secure this connection, SSH is configured so that the SSH Transport Layer Protocol establishes a TCP connection between the SSH client and server entities, with TCP port numbers *a* and *b*, respectively.
- A secure SSH tunnel is established over this TCP connection.
- Traffic from the client at port *x* is redirected to the local SSH entity and travels through the tunnel where the remote SSH entity delivers the data to the server application on port *y*.
- Traffic in the other direction is similarly redirected.
- SSH supports two types of port forwarding: local forwarding and remote forwarding.

Local forwarding allows the client to set up a "hijacker" process. This will intercept selected application-level traffic and redirect it from an unsecured TCP connection to a secure SSH tunnel. SSH is configured to listen on selected ports.

Example: Suppose you have an e-mail client on your desktop and use it to get e-mail from your mail server via the Post Office Protocol (POP). The assigned port number for POP3 is port 110. We can secure this traffic in the following way:

1. The SSH client sets up a connection to the remote server.
2. Select an unused local port number, say 9999, and configure SSH to accept

traffic from this port destined for port 110 on the server.

3. The SSH client informs the SSH server to create a connection to the destination, in this case mailserver port 110.
4. The client takes any bits sent to local port 9999 and sends them to the server inside the encrypted SSH session. The SSH server decrypts the incoming bits and sends the plaintext to port 110.

In the other direction, the SSH server takes any bits received on port 110 and sends them inside the SSH session back to the client, who decrypts and sends them to the process connected to port 9999.

With **remote forwarding,** the user's SSH client acts on the server's behalf. The client receives traffic with a given destination port number, places the traffic on the correct port and sends it to the destination the user chooses.

Example: You wish to access a server at work from your home computer. Because the work server is behind a firewall, it will not accept an SSH request from your home computer. However, from work you can set up an SSH tunnel using remote forwarding.

- This involves the following steps.
  1. From the work computer, set up an SSH connection to your home computer. The firewall will allow this, because it is a protected outgoing connection.
  2. Configure the SSH server to listen on a local port, say 22, and to deliver data across the SSH connection addressed to remote port, say 2222.
  3. You can now go to your home computer, and configure SSH to accept traffic on port 2222.
  4. You now have an SSH tunnel that can be used for remote logon to the work server.

Assessment questions to the lecture

Qn No	Question	Answer	Bloom's Knowledge Level
1	What does HTTPS encrypt during communication? a) URL of the requested document b) Contents of browser forms c) Cookies sent between the browser and server d) All of the above	d	Remembering
2	Which RFC documents the HTTPS protocol? a) RFC 4251 b) RFC 2818 c) RFC 2409 d) RFC 4250	b	Remembering

3	What is the primary purpose of port forwarding in SSH? a) Encrypting secure email communications b) Securing TCP connections by tunneling traffic through SSH c) Allowing unauthorized access to remote servers d) Multiplexing multiple SSH connections	b	Remembering
4	Which hash function is decided during the SSH algorithm negotiation? a) MD5 b) SHA c) The hash function is chosen based on the agreed algorithm d) CRC32	c	Remembering

Students have to prepare answers for the following questions at the end of the lecture

Qn No	Question	Marks	CO	Bloom's Knowledge Level
1	What is Secure Shell (SSH) and what is its application?	2	3	Remembering
2	Mention two elements of communication encrypted in HTTPS.	2	3	Remembering
3	Explain the architecture of HTTPS, including how it ensures secure communication between a client and server. Discuss the TLS handshake process and its role in HTTPS.	16	3	Understanding
4	Explain the SSH protocol stack in detail with a neat diagram. Explain the SSH user authentication protocol and connection protocol with the steps involved in message exchanges. (April/May 2024)	16	3	Understanding

Reference Book:

Author(s) Name	Title of the book	Page numbers
William Stallings	Cryptography and Network Security: Principles and Practice, 6th Edition	543-555